

1 – Notions de bases sur les SE

Historique

Processeur

Environnement d'Exécution

Système d'Exploitation

UTC502

2 – Gestion de l'Exécution des Programmes

Processus, Ordonnancement, Threads

Approche de la Multiprogrammation et de ses Risques

3 – Gestion de la mémorisation

Les stratégies d'allocation

Gestion paginée de la MC

4 – Administration de Système

Droits, Comptes ...

TD : Programmation Système sous Windows

- Appels systèmes

- Processus, Threads

- Initiation à la programmation parallèle

Pourquoi apprendre comment fonctionne les ordinateurs ?

Indépendamment d'un simple aspect de curiosité intellectuelle, la compréhension des concepts est indispensable pour l'ingénieur.

Il faut comprendre les principes sous-jacents si l'on veut concevoir des applications efficaces.

Il existe des logiciels plus efficaces que d'autres, non seulement parce que leur auteur a trouvé une méthode algorithmique plus subtile mais aussi parce qu'il a su mieux respecter les principes cachés dans le système d'exploitation de la machine et tirer le meilleur parti du fonctionnement du processeur et de la mémoire.

Ceci est encore plus vrai avec les nouvelles générations de machines massivement « parallèles ».



Avant-Propos

A l'heure actuelle le système d'exploitation prend une importance grandissante sur le fonctionnement de nos systèmes d'information.

*Ce qui fait la spécificité des systèmes d'exploitation, c'est **la pression continue des utilisateurs toujours plus nombreux et exigeants.***

*Ils imposent aux concepteurs des OS d'étendre continuellement leurs fonctionnalités tout en gardant un souci d'efficacité et de sécurité maximale **en se projetant dans l'avenir.***

*L'esprit des Systèmes d'Exploitation consiste en la recherche du **meilleur compromis entre fonctionnalité, performance, sécurité et maintenabilité.***

Il demande de la rigueur dans l'analyse des problèmes, mais aussi de l'inventivité et de l'astuce dans l'implémentation.

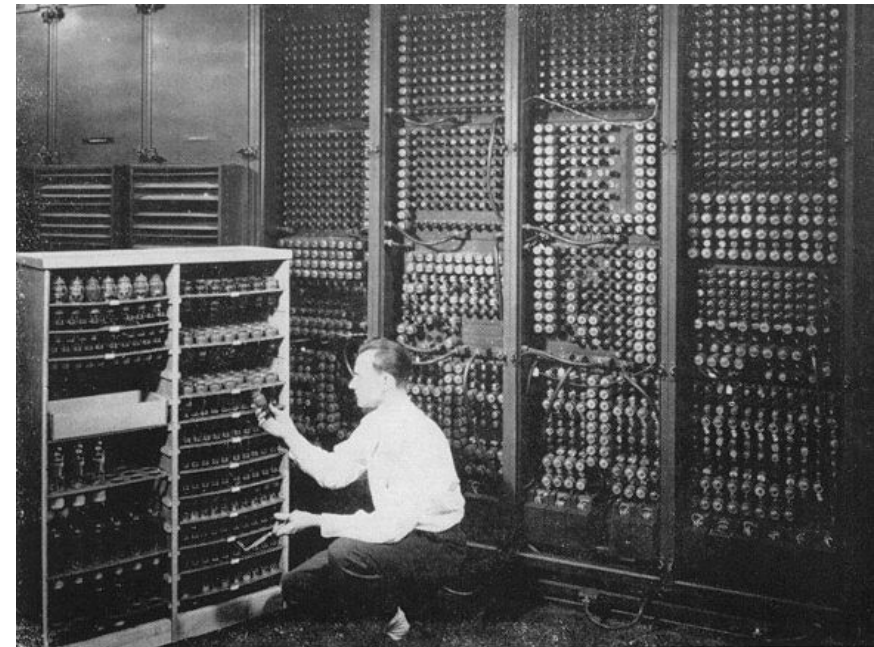


1.1 – Historique – 1ère génération

1936 -1955 Programmation en langage machine

Pas d'OS

Les programmes étaient écrits en langage machine en basculant des tableaux d'interrupteurs pour contrôler les fonctions de base de la machine.



Replacing a bad tube meant checking among ENIAC's 19,000 possibilities.



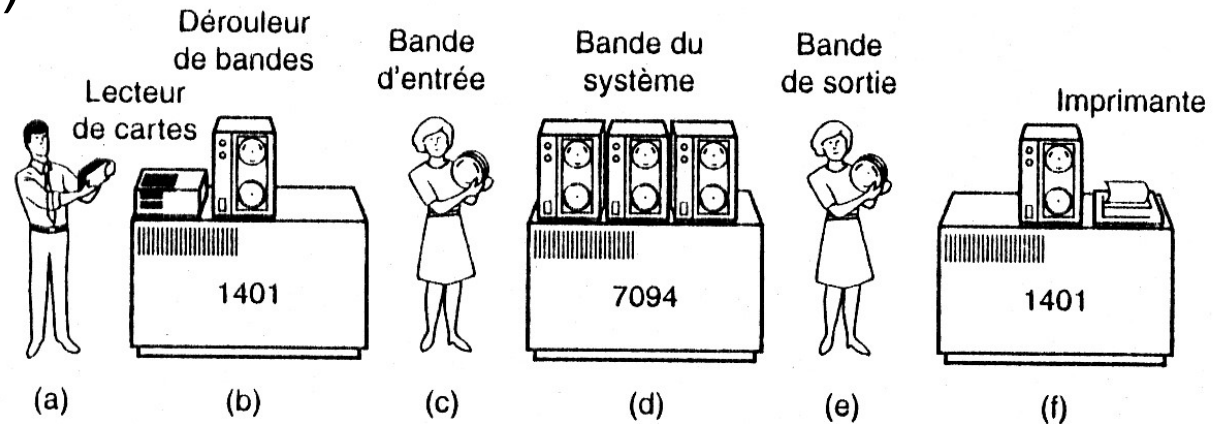
*1950 - Invention du transistor
Cartes perforées*



1.1 – Historique – 2^{ème} génération

1955 -1965 Langage d'Assemblage premier langage évolué
(Fortran)

Traitement par lots



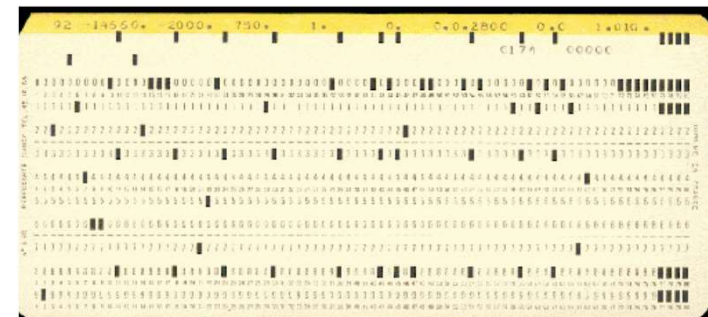
- (a) Les programmeurs apportent leurs cartes au 1401.
(b) Le 1401 lit l'ensemble des jobs sur bande.
(c) L'opérateur apporte la bande au 7094.
(d) Le 7094 effectue les calculs demandés.
(e) L'opérateur transfère les résultats sur le 1401.

Les SE :

- FMS (Fortran Monitor System)
- IBSYS (SE IBM7094)



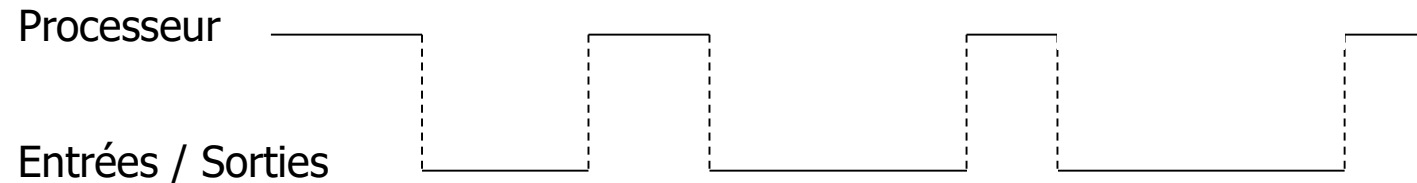
1960 - *Invention des Circuits intégrés*
Apparition des Interruptions



1.1 – Historique – 3^{ème} génération

1965 -1980 Interaction utilisateurs/système

Multiprogrammation - Temps Partagés - Transactionnel



Utilisation du processeur lors de l'exécution d'un programme

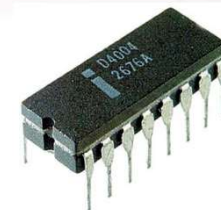
Les systèmes se nomment :

1965	MULTICS
1967	OS/360 d'IBM
1969	UNIX



1974 – Intel sort le 8080 premier processeur 8bits

*1980 - Développement des Circuits LSI
(Circuits intégrés à haute densité)*



1 – Historique – 4^{ième} génération

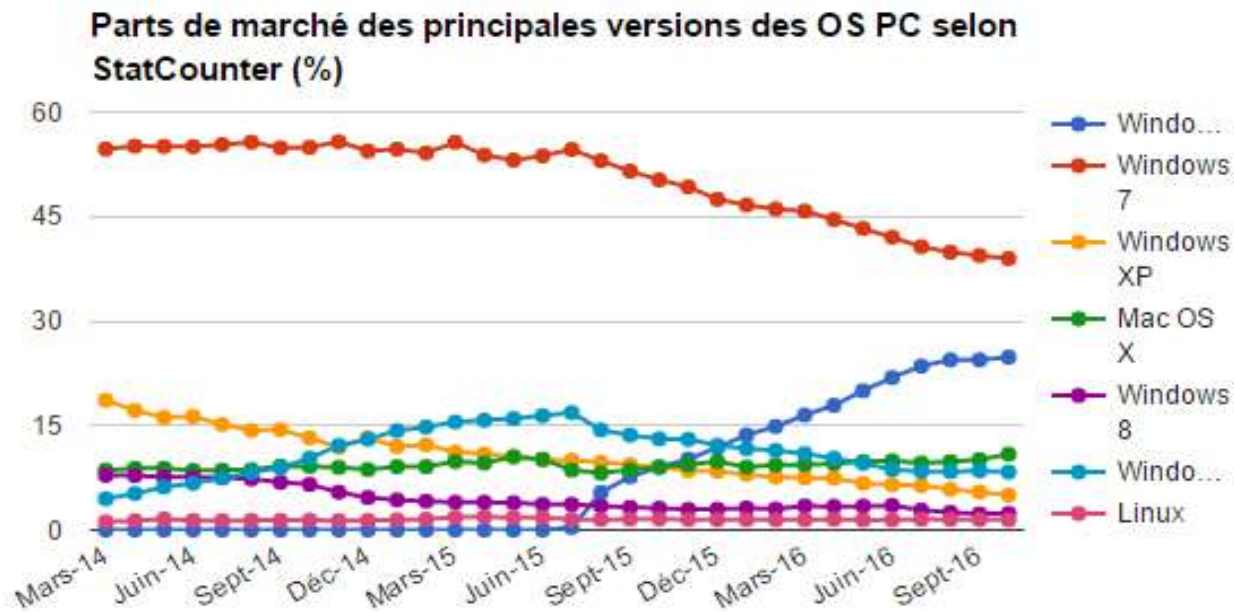
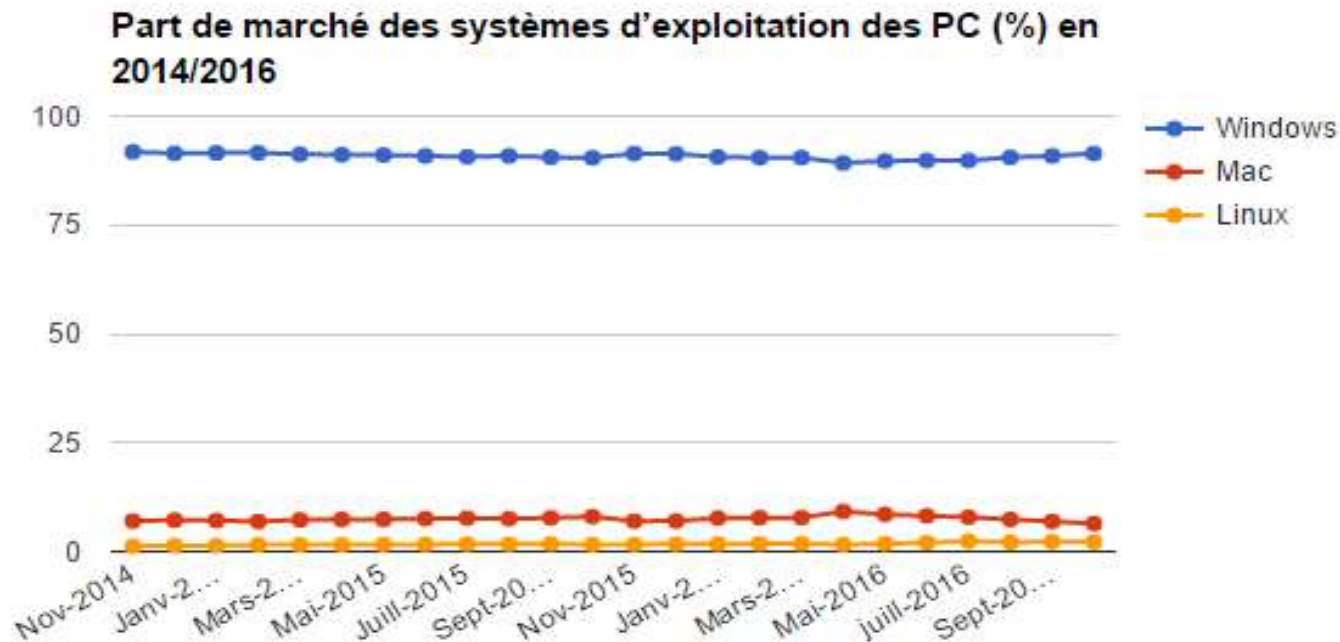
1980 Ordinateurs personnels -.

Réseaux – Clients Serveur – Système distribué

1976	CP/M (Digital Research)			
1977	VMS (Digital Research)			
1981	MS-DOS (Microsoft)			
1984	MAC OS (Apple iOS)			
1985	OS/2 (IBM+Microsoft)			
	Windows 1.0			
1990	Windows 3.0			
1991	LINUX			
1993	Windows NT			
1995	Windows 95			
2000	Windows 2000			
2002	Windows XP			
2006	Windows Vista			
2009	Windows Seven			
2012	Windows 8	iOS 6	Ubuntu 12-Debian 6	Android 4
2015	Windows 10			



1.2 – Les SE des PC actuellement ...



OS Serveurs :

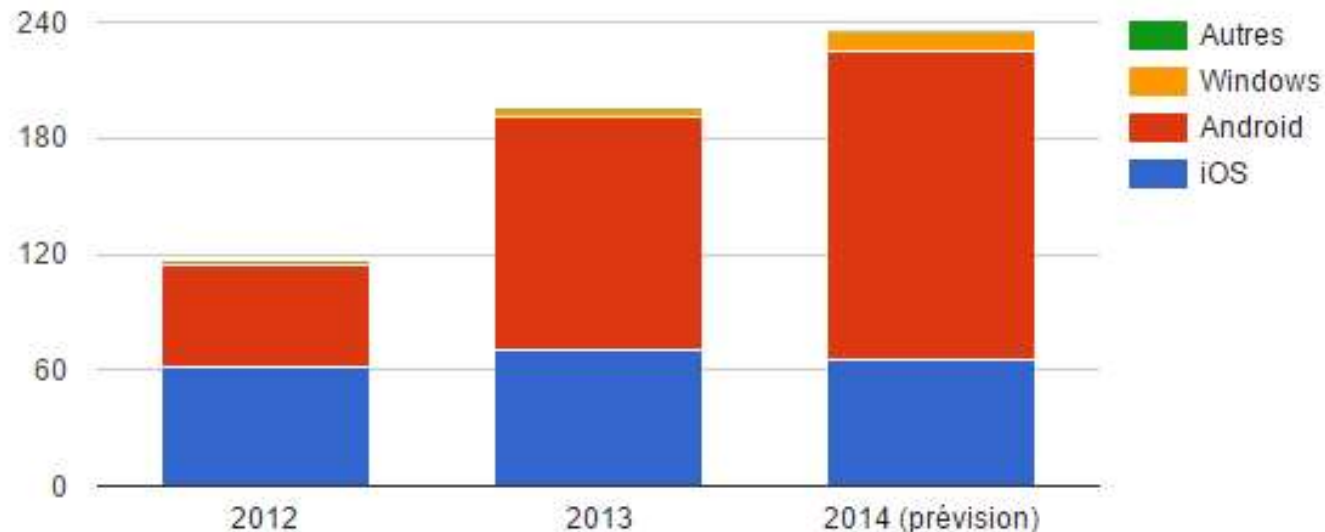
Windows 30 %

Unix-Linux 38 %

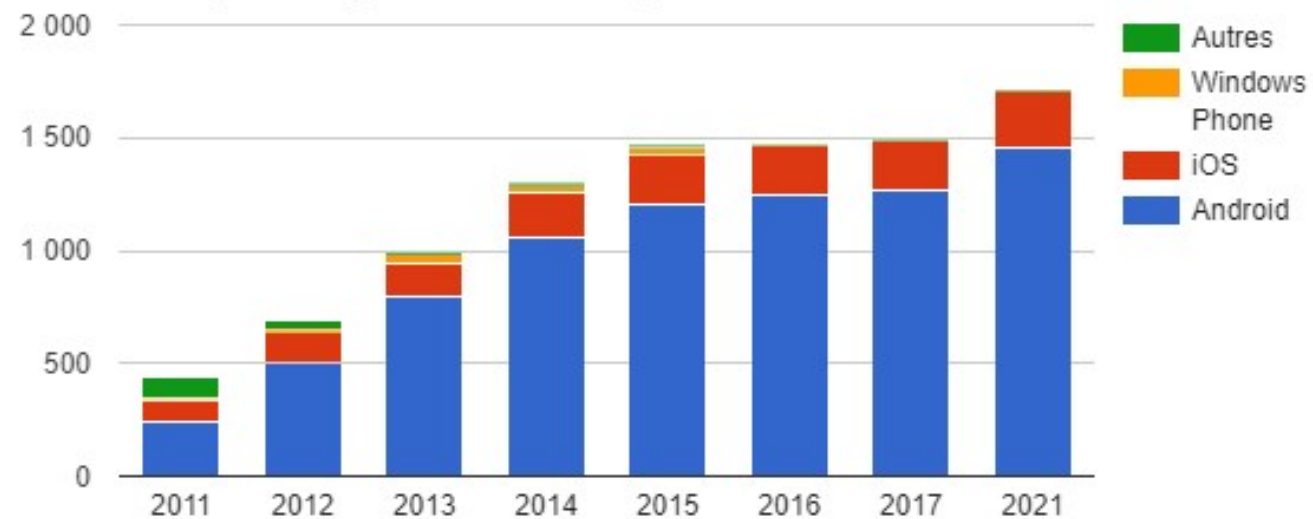


1.2 – Les SE des mobiles actuellement ...

Tablettes livrées dans le monde par OS (millions d'unités)



Répartition par OS des livraisons mondiales de smartphones (millions d'unités)



1.2 – Définition du Système d'Exploitation

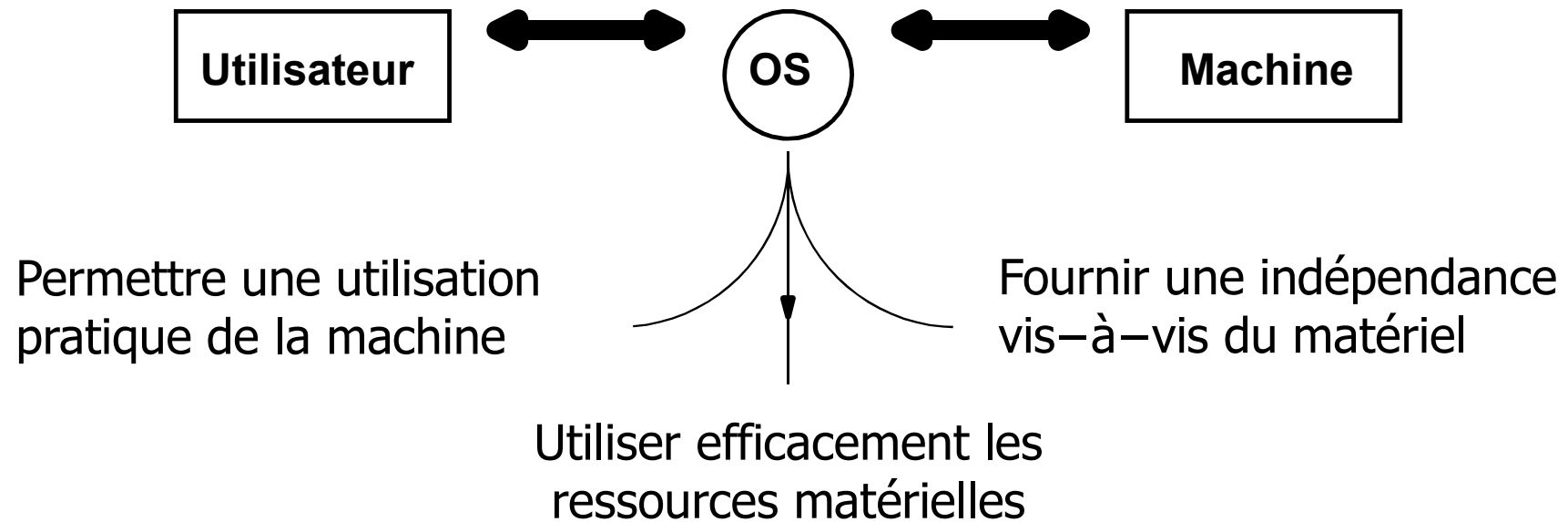
Définition :

Un système d'exploitation est un ensemble de fonctions, mises en œuvre par logiciel, permettant d'offrir à ses utilisateurs une version plus abstraite de la machine physique : la machine virtuelle.

La machine virtuelle est définie par une interface, composée d'un ensemble d'appels système.



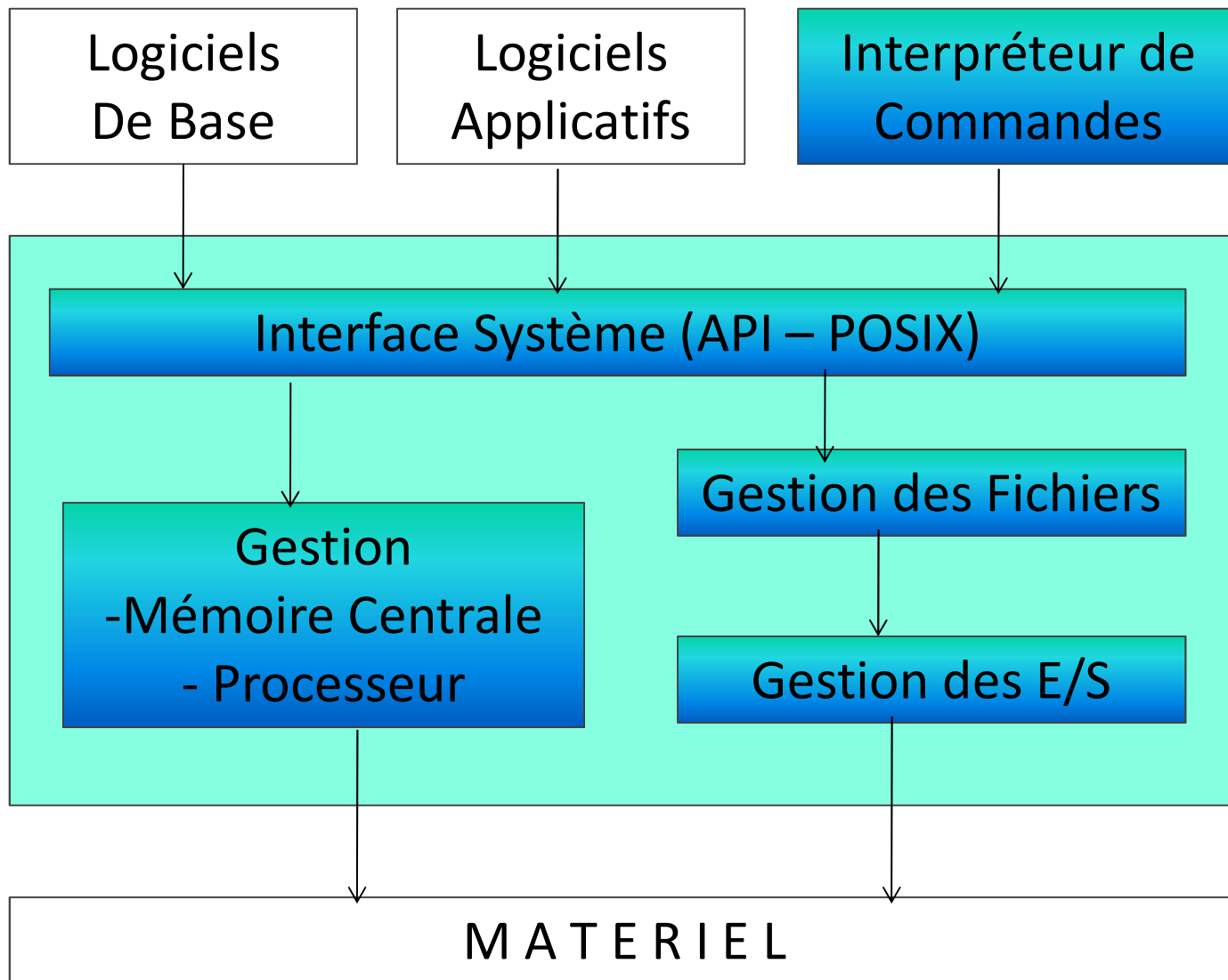
1.3 – Rôles du Système d'Exploitation



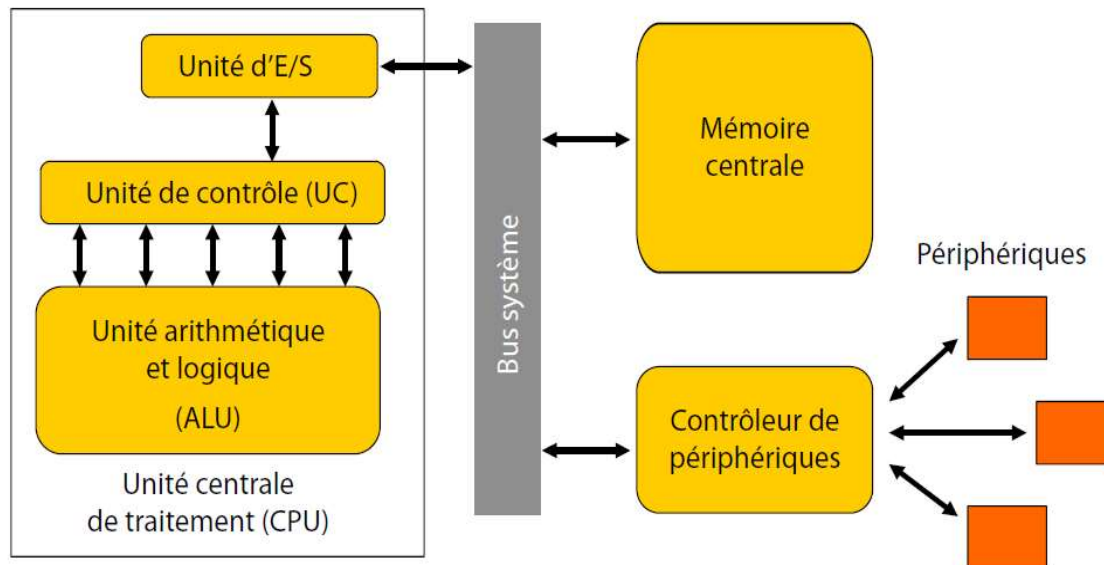
- prendre en charge la gestion complexe des ressources de l'ordinateur
- distribuer les ressources de façon cohérente entre les différents processus
- assurer la sécurité de l'ensemble des processus
- fournir une machine virtuelle à l'utilisateur



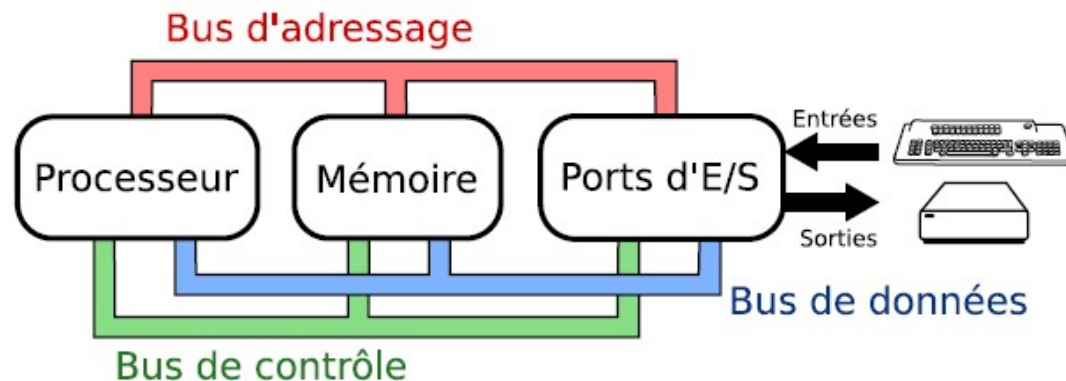
1.4 – Schéma : Exemple d'un Système d'Exploitation



1.5 – Structure Matérielle de l'Ordinateur



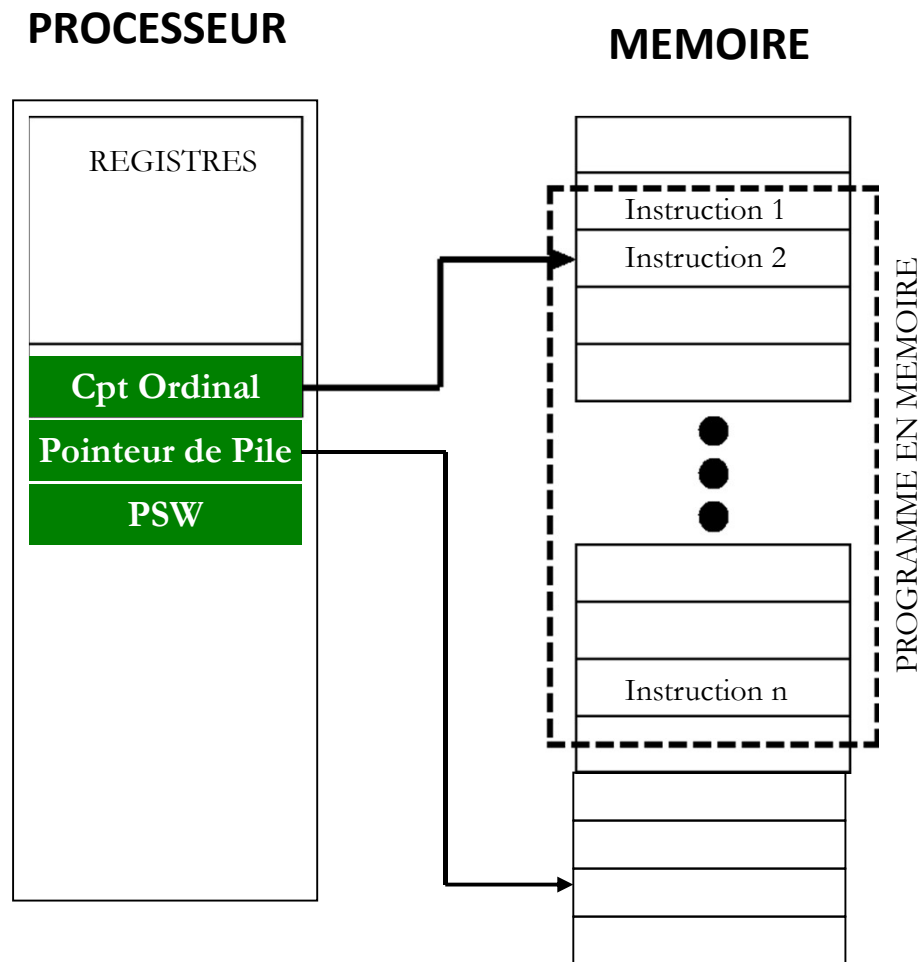
Le bus est le circuit d'échange des informations. Son débit ou bande passante est un paramètre important pour définir la rapidité d'une machine. Souvent, dans les micro-ordinateurs, elle est trop faible par rapport à la rapidité du processeur. Celui-ci attend fréquemment les informations en provenance de la mémoire ou des contrôleurs d'entrées-sorties.



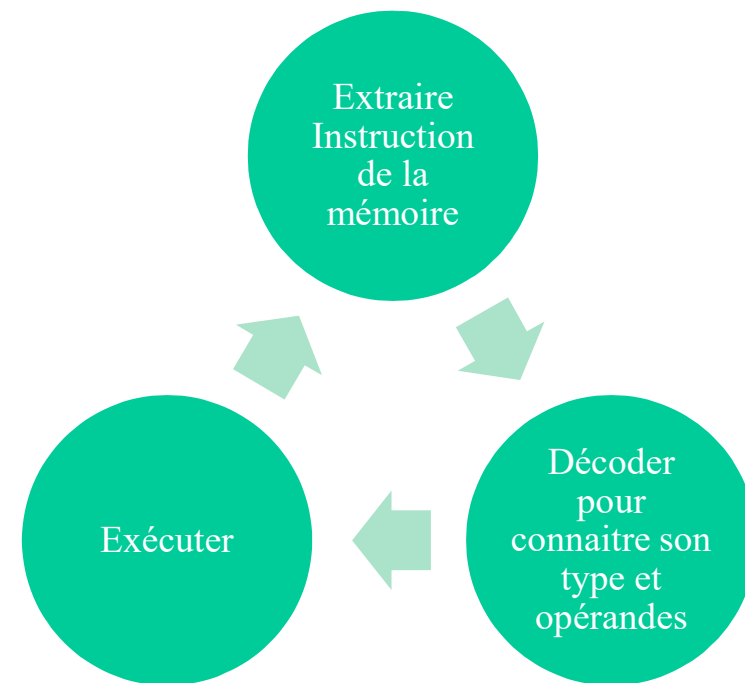
Aujourd'hui les machines ont une structure composée de plusieurs bus.



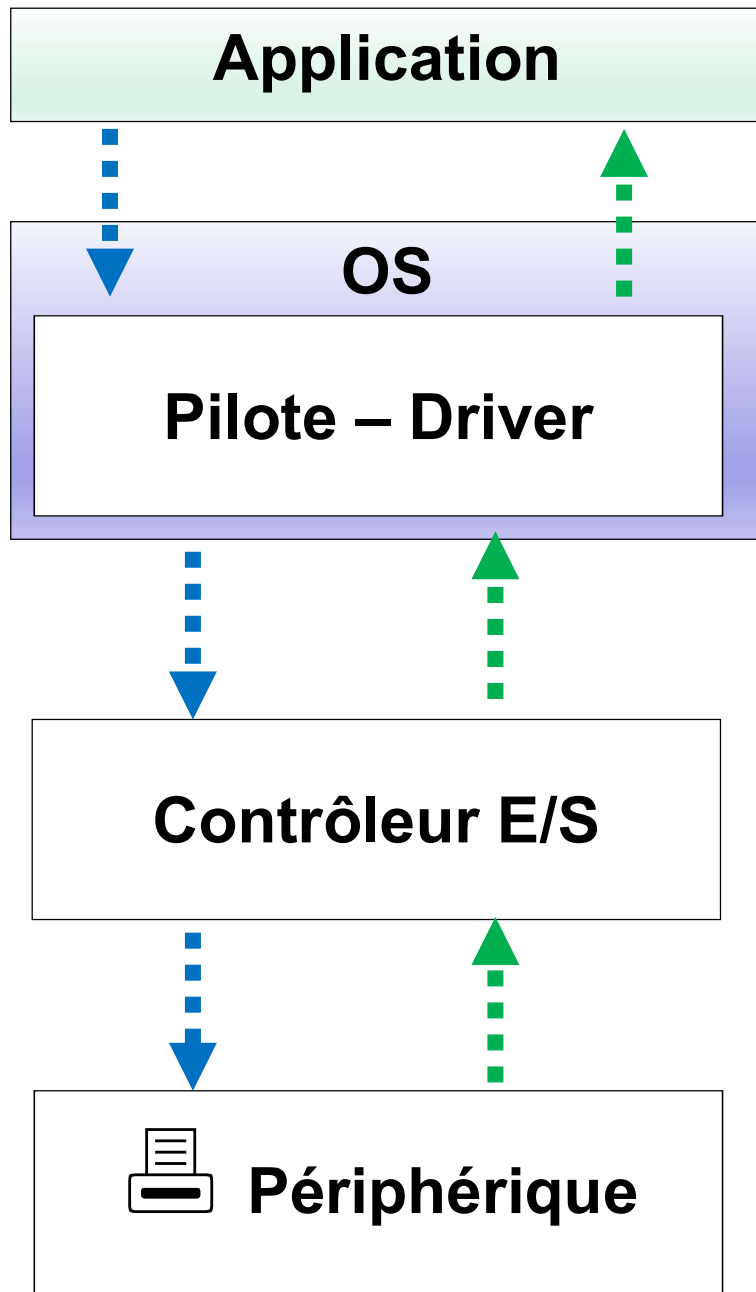
1.5.1 – Le Processeur



Le cycle de base de la CPU



1.5.2 – Les Entrées / Sorties (I/O)

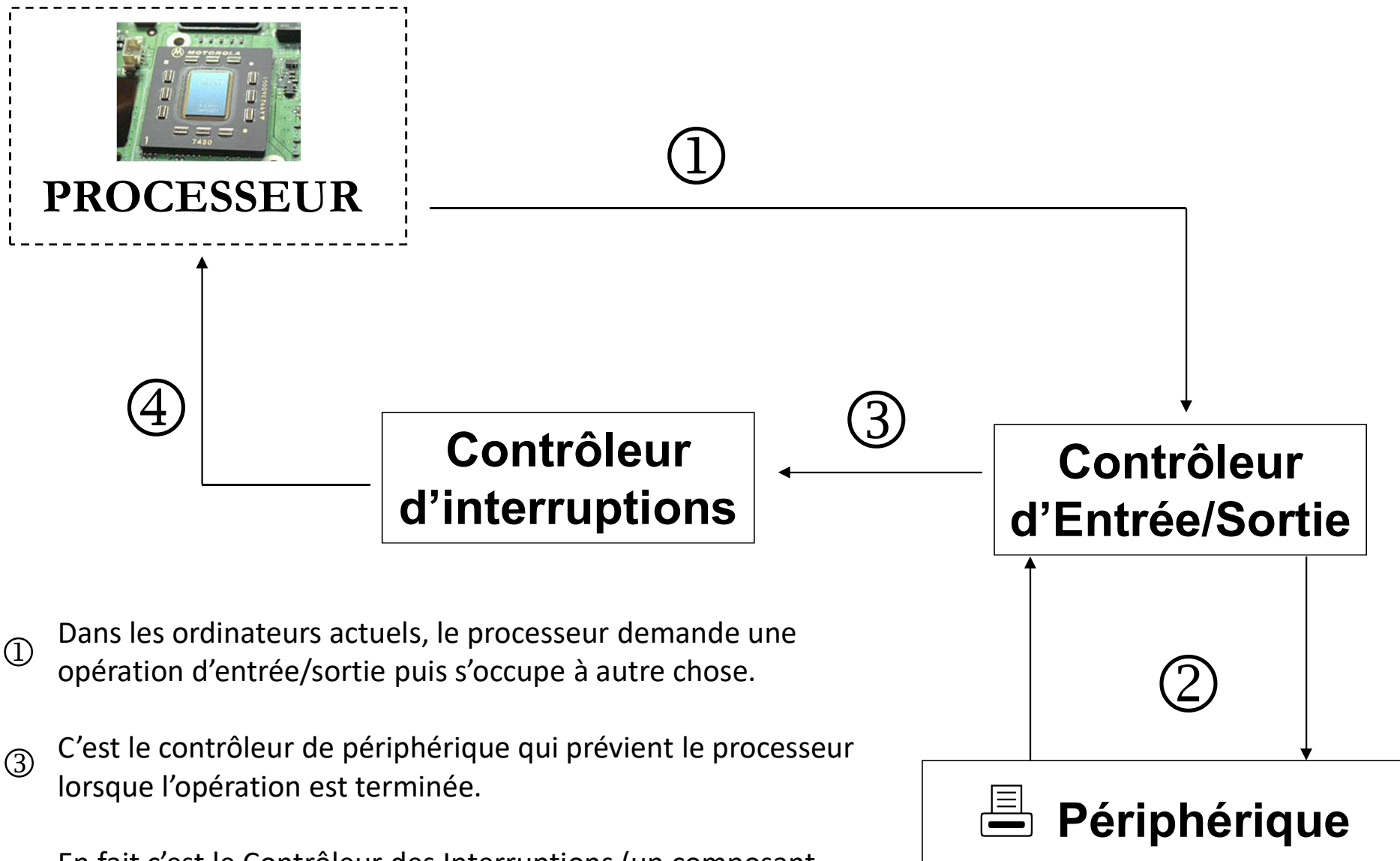


- Appartient à l'OS
- Effectue des requêtes d'E/S par positionnement des registres
- Signale la fin d'une E/S par une interruption

- Gérer plusieurs périphériques de même type
- Harmoniser leur comportement
- Offrir une interface simple à l'OS



1.5.3 – Les Interruptions



- ① Dans les ordinateurs actuels, le processeur demande une opération d'entrée/sortie puis s'occupe à autre chose.
- ③ C'est le contrôleur de périphérique qui prévient le processeur lorsque l'opération est terminée.
- ④ En fait c'est le Contrôleur des Interruptions (un composant particulier) qui décide ou non de transmettre l'interruption au processeur.

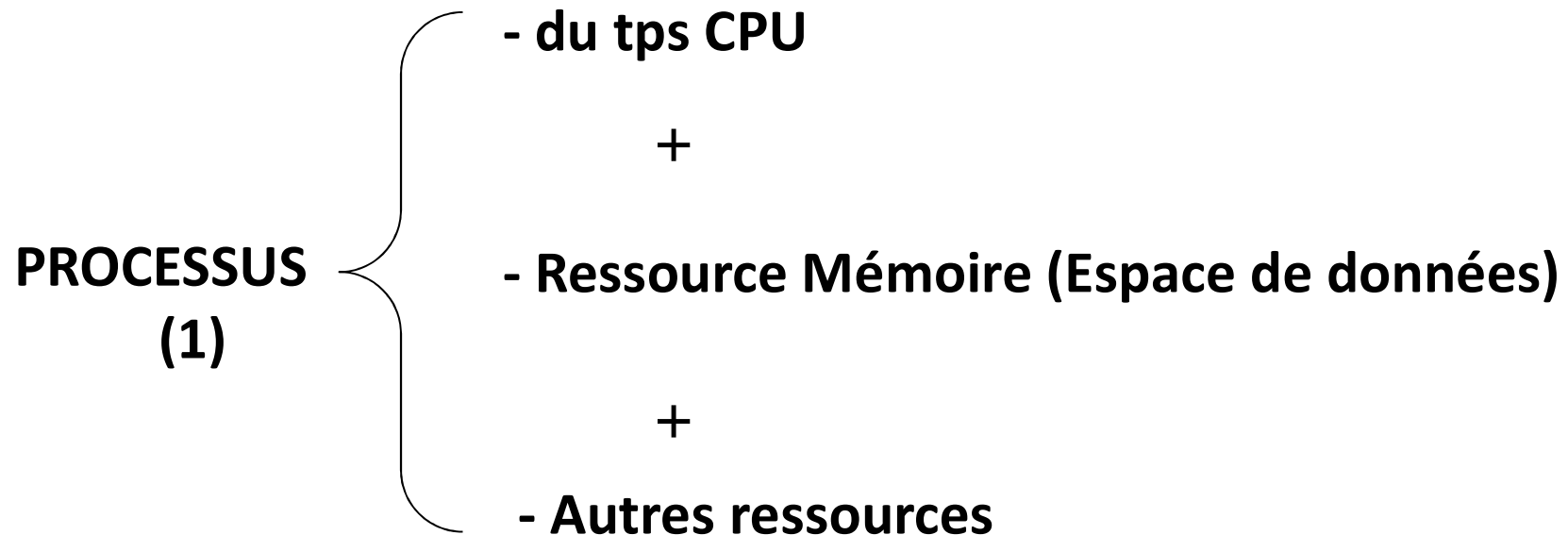


2 – Notion de processus

DEFINITION

Un processus est une instance dynamique d'un programme en langage machine.

Un Processus réclame :



2 – Notion de processus

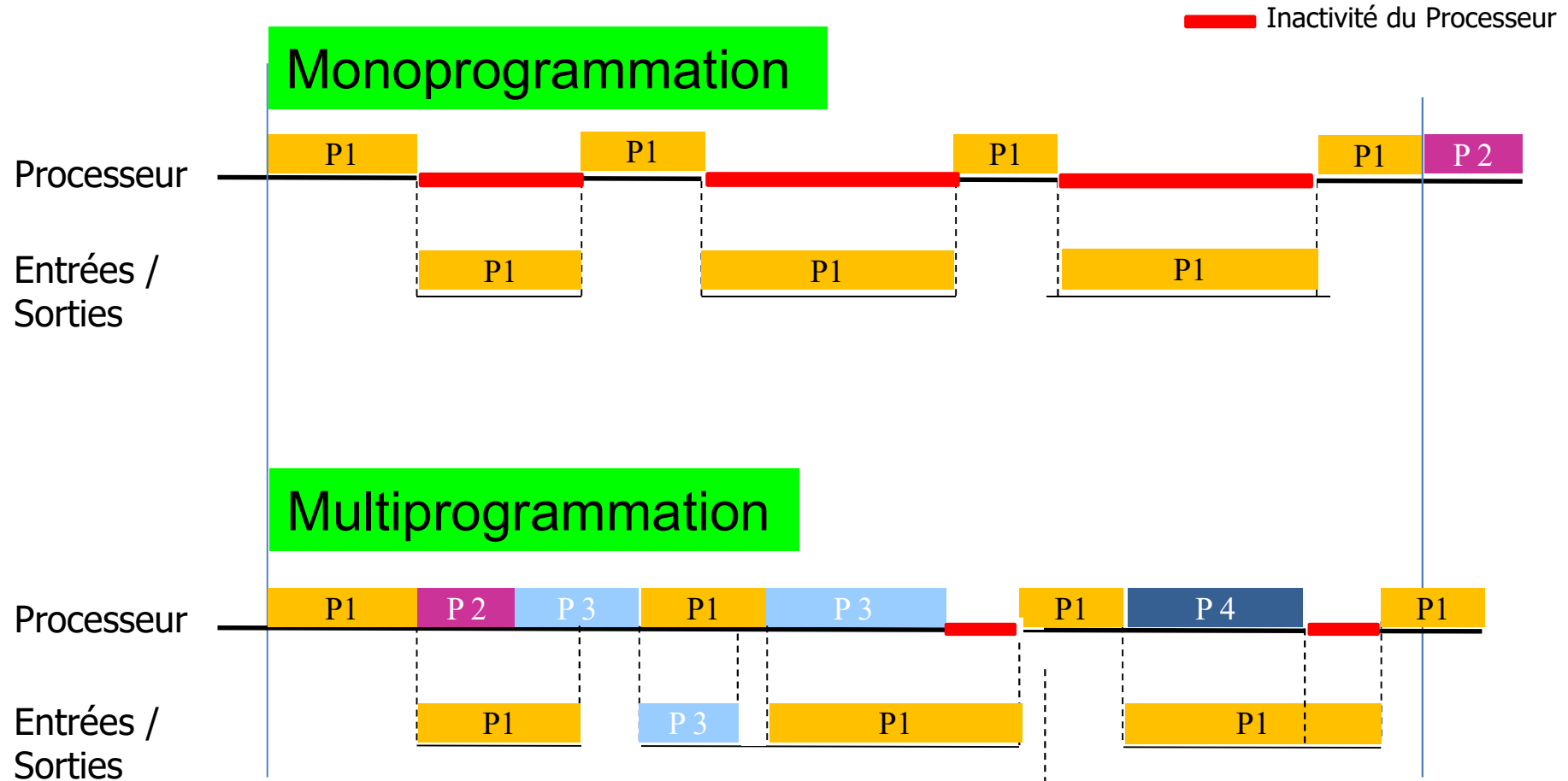
Un seul processeur exécute combien de processus en même temps ?



L'illusion que la machine exécute plusieurs programmes en simultanés est possible grâce à quoi ?

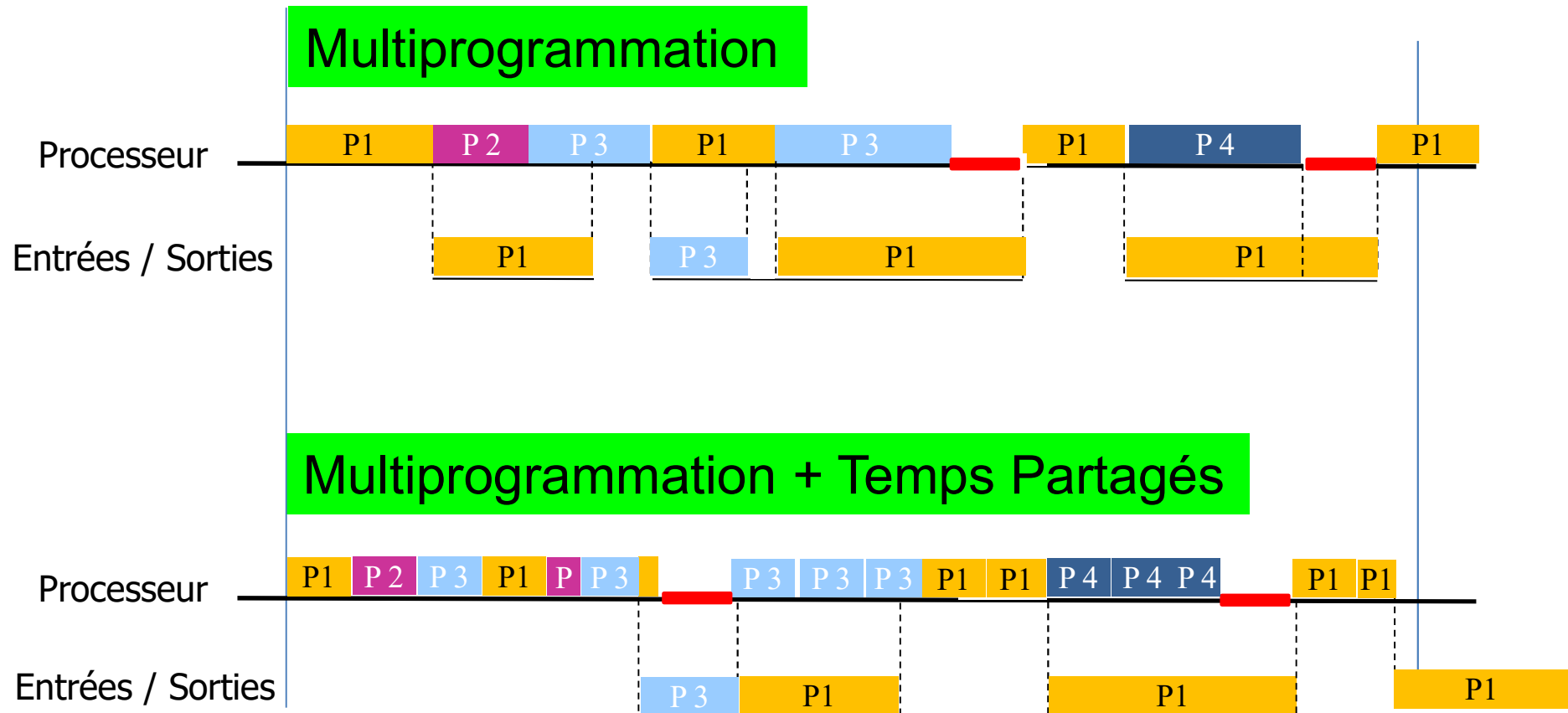


2.2 – De la Monoprogrammation à la Multiprogrammation

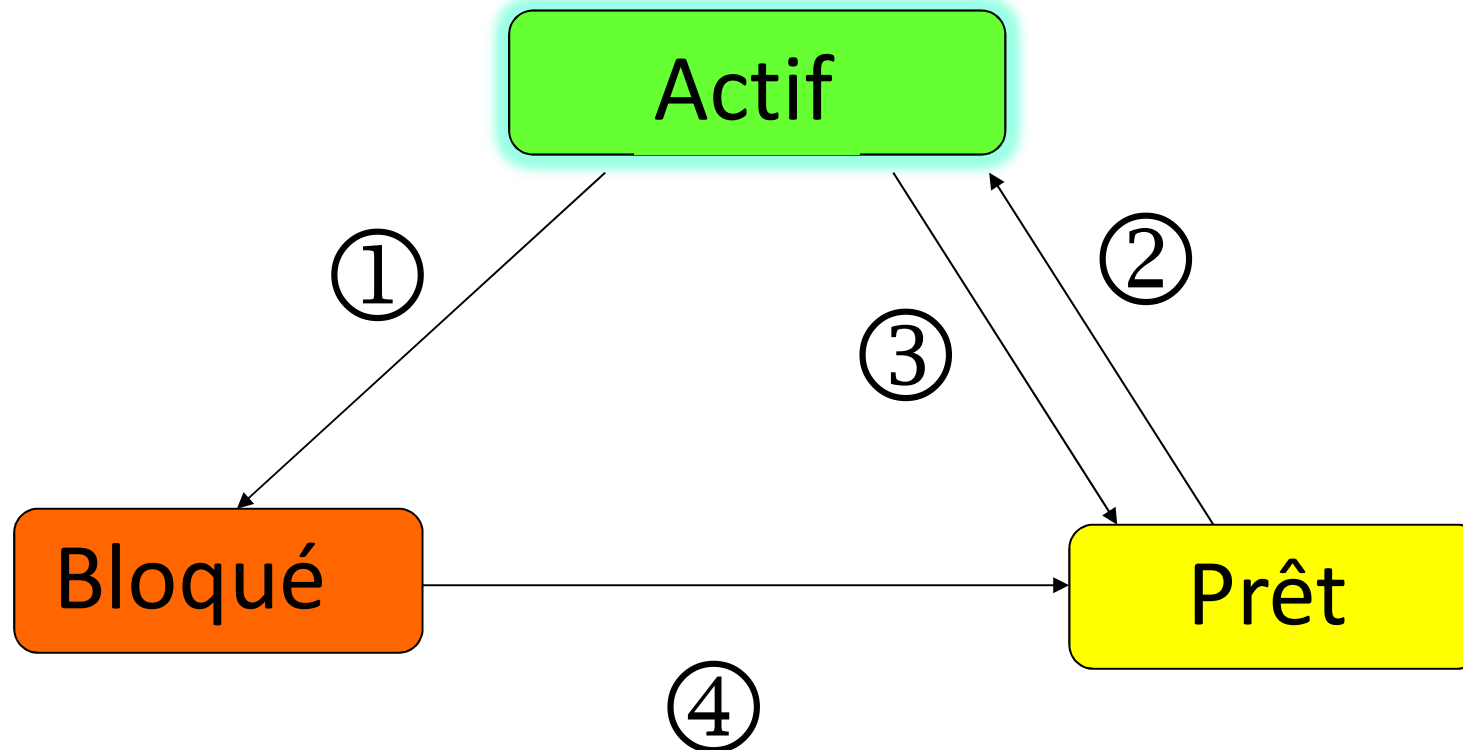


2.2 – De la Monoprogrammation à la Multiprogrammation

■ Inactivité du Processeur



2.2 – Notion de processus : Les Etats



2.3 – Notion de processus : PCB

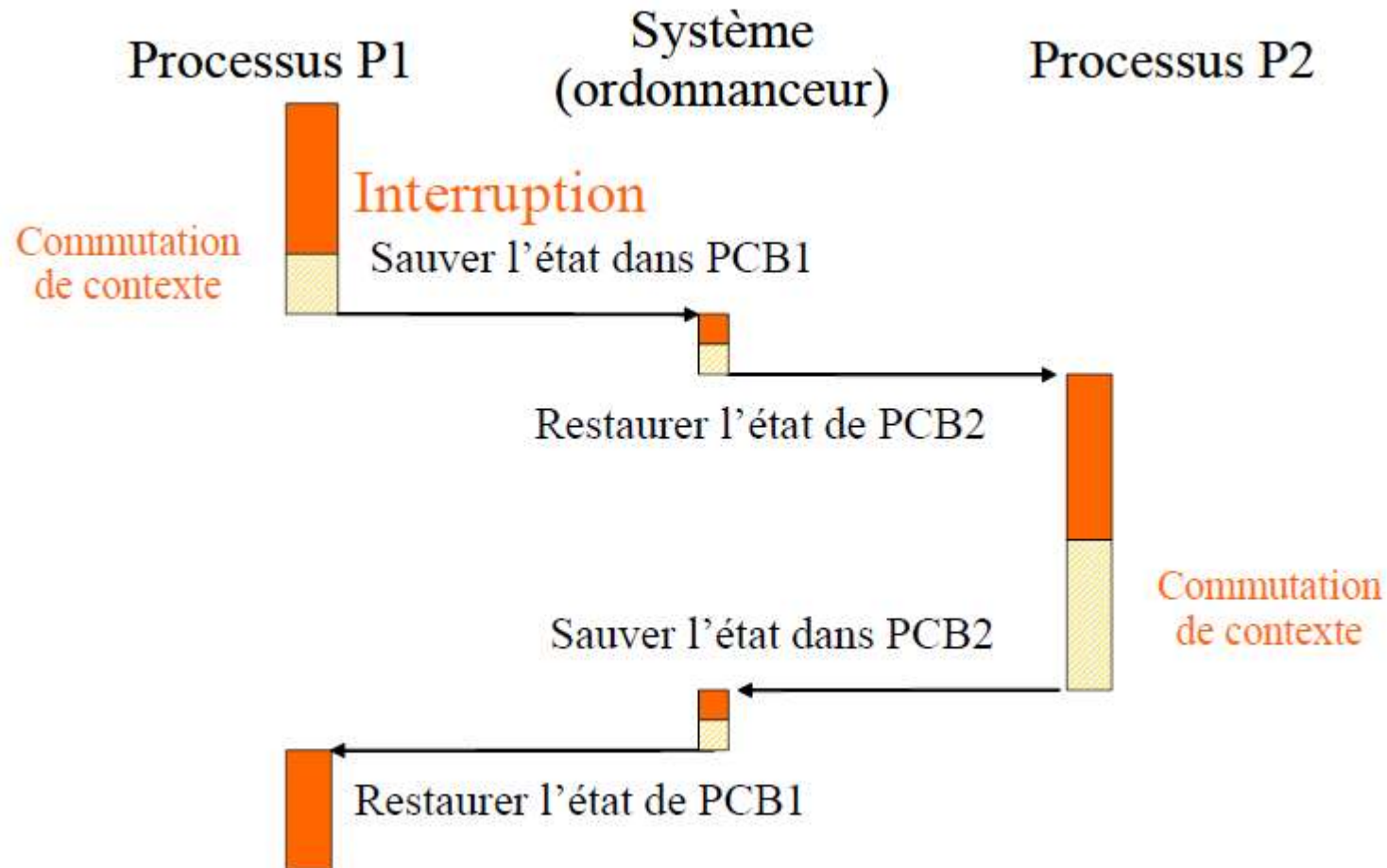
Chaque processus est représenté dans le SE par un bloc de contrôle (PCB).

Dans le PCB, le SE stocke des informations pouvant varier d'un processus à l'autre.

Pointeur père (<i>ppid</i>)	État du processus
Numéro du processus (<i>pid</i>)	
Compteur ordinal	
Registres	
Limites mémoire	
Fichiers ouverts	

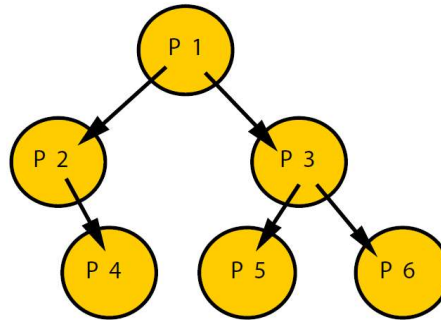


2.4 – Notion de processus : Commutation de contexte



2.5 – Notion de processus UNIX: Hiérarchie des processus

Sous **Unix** un processus et l'ensemble de ces descendants forment un groupe. Lorsqu'un utilisateur envoie un signal clavier, celui-ci est délivré à tous les membres du groupe. Individuellement tous les processus peuvent intercepter le signal et l'ignorer.



Cas de terminaison

- Exécution de la dernière instruction
- Appel système
 - depuis le processus lui-même
 - depuis un autre processus (souvent le père)
- Terminaison du père (terminaison en cascade)



2.5 – Processus avec Unix et Linux

On peut à un moment donné avoir plusieurs processus en cours. Le système doit être capable de les identifier. Pour cela, il attribue à chacun d'entre eux, un numéro unique, appelé **PID** (Process Identification).

Un processus peut lui-même créer d'autres processus (on dit qu'il *fork*). Il devient donc un processus parent ou père, et chaque nouveau processus s'appelle un processus *enfant*.

Ce dernier est identifié de manière unique par son PID, et son processus père par le **PPID** (Parent Process Identification), qui est en fait le PID de ce processus père.

Oui mais dans tout ça, qui a créé le premier processus ?



*Le seul qui ne suit pas cette règle est le premier processus lancé sur le système : le **processus init**,
Il est lancé directement par le noyau, pendant le boot.*



2.5 – Processus avec Unix et Linux : fork()

- fork() crée un nouveau processus (clonage de processus)
- fork() duplique l'espace d'adressage du processus original (code, données, variables, registres, descripteurs de fichiers, ...)
- Le processus appelant la fonction fork() est le processus père (parent process)
- Le processus créé est le processus fils (child process)



Exemple LINUX



2.6 – Notion de processus : Windows pas de hiérarchie ...

Windows n'intègre pas le concept de hiérarchie de processus. Tous les processus sont égaux.

Le seul moment où l'on trouve une certaine hiérarchie est celui de la création d'un processus.

Le parent peut récupérer le « handle » du processus fils créé et peut l'utiliser pour le contrôler.

handle : Identifiant système



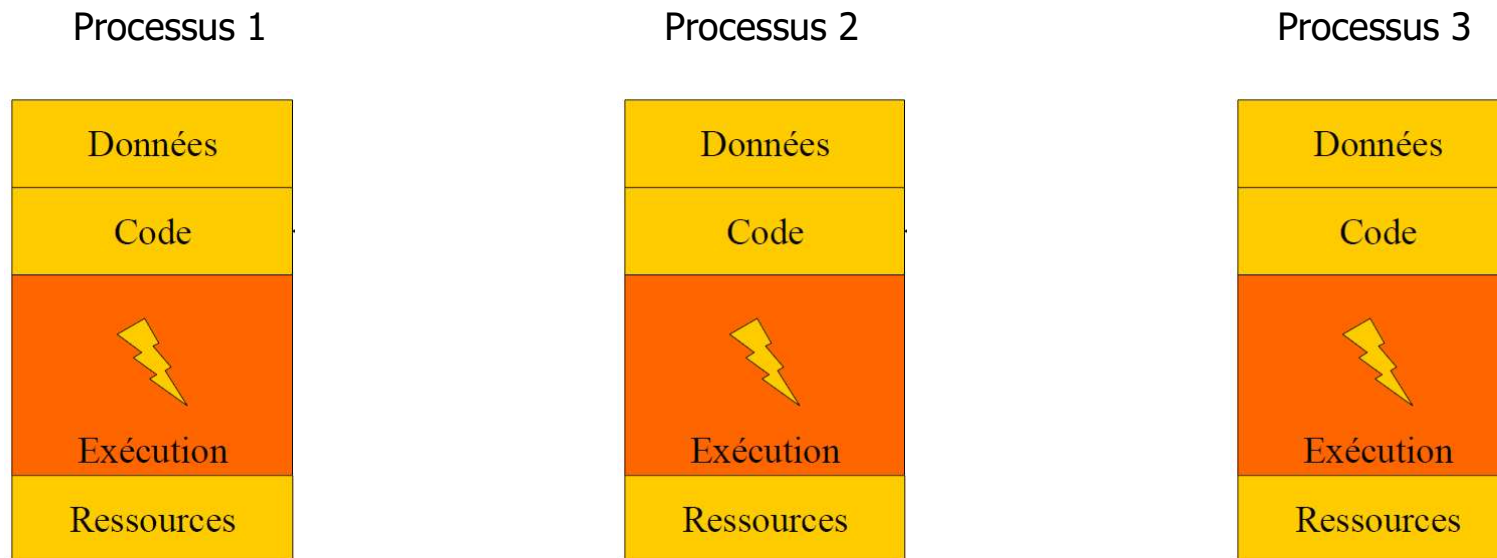
Exemple Windows



2.7 – Notion de Thread

Dans un contexte « classique » chaque processus possède un espace d'adressage et un thread unique.

Processus concurrents



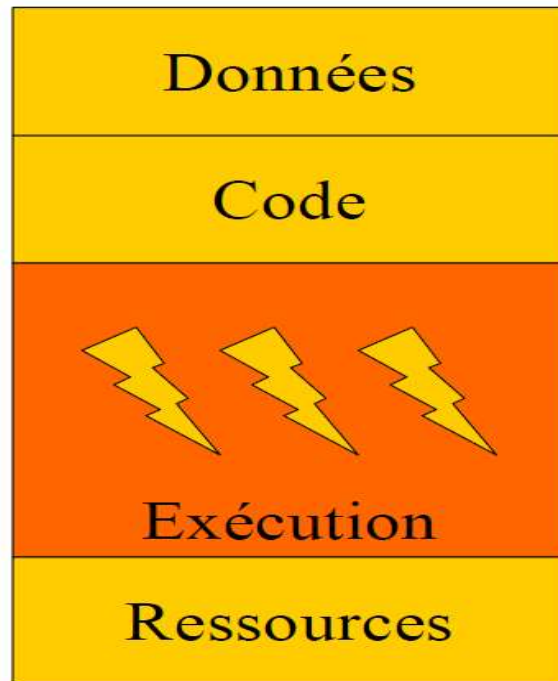
1 seul fil d'exécution (Thread) dans chacun des processus



2.7 – Notion de Thread : Processus Multithread

Il arrive qu'un processus ait besoin de plusieurs threads dans le même espace d'adressage.

Processus Multithread



Quel autre nom est utilisé pour désigner un thread ?



Processus Légers

3 Thread dans le même processus



2.8 – Notion de Thread : Implémentation

API Linux

```
#include <pthread.h>
pthread_t thread1, thread2;
char * message1 = "Thread 1";
int    iret1;

iret1 = pthread_create (& thread1, NULL, Procedure, ( void *) message1);
```

API Windows

```
DWORD ThreadId;
HANDLE hdl_thread1;

hdl_thread1=CreateThread(NULL,NULL,Procedure,NULL,NULL,&ThreadId);
if (hdl_thread1 == NULL)
{
    printf("le thread n'a pu etre lancé\n");
}
```

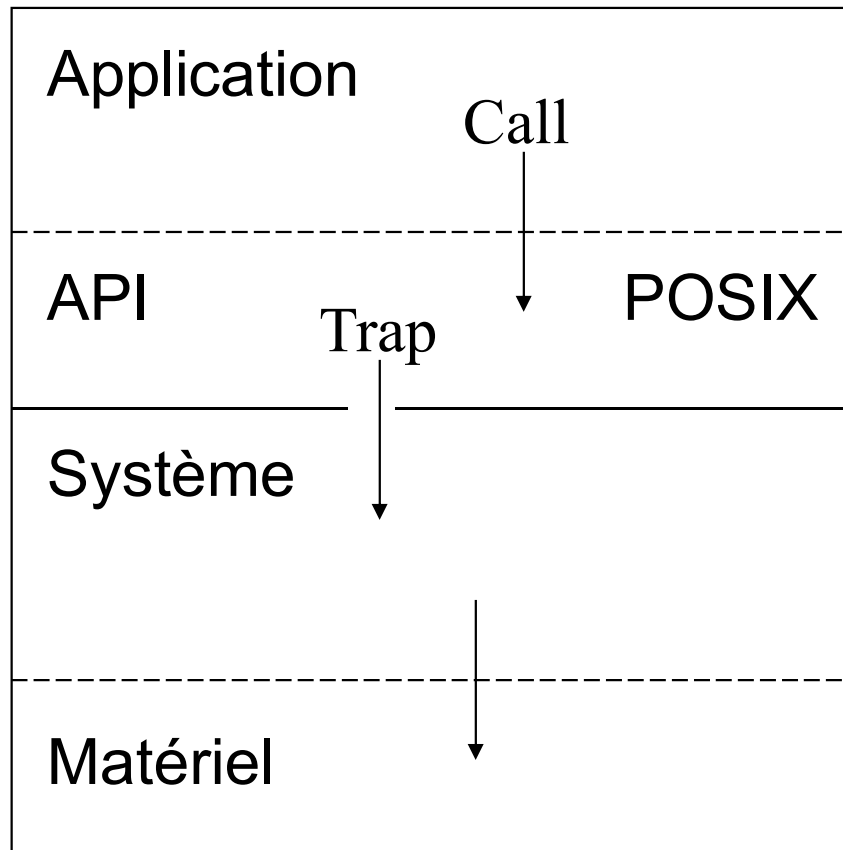


Exemple Windows



2.9 – Les Appels Systèmes

Ils constituent l'interface entre le système d'exploitation et les programmes d'application.



2.9 – Les Appels Systèmes Unix et Win32

UNIX	Windows	
	CreateProcess	Crée un nouveau processus
waitpid	WaitForSingleObject	Attend la Fin d'un processus
exit,kill	TerminateProcess	Termine l'exécution d'un processus
open	CreateFile	Crée un nouveau fichier
close	CloseHandle	Ferme un fichier
read	ReadFile	Lit des données dans un fichier
write	WriteFile	Ecrit des données dans un fichier
Pthread_create	CreateThread	Création d'un thread
exec		Remplace l'image du processus actuel par un autre processus
fork		Duplique le processus



2.10 – ORDONNANCEMENT (Gestion d'une File d'Attente)

SITUATIONS DE CONCURRENCE

↳ L'allocateur de ressource (l'ordonnanceur)

CHARGE DU SYSTEME

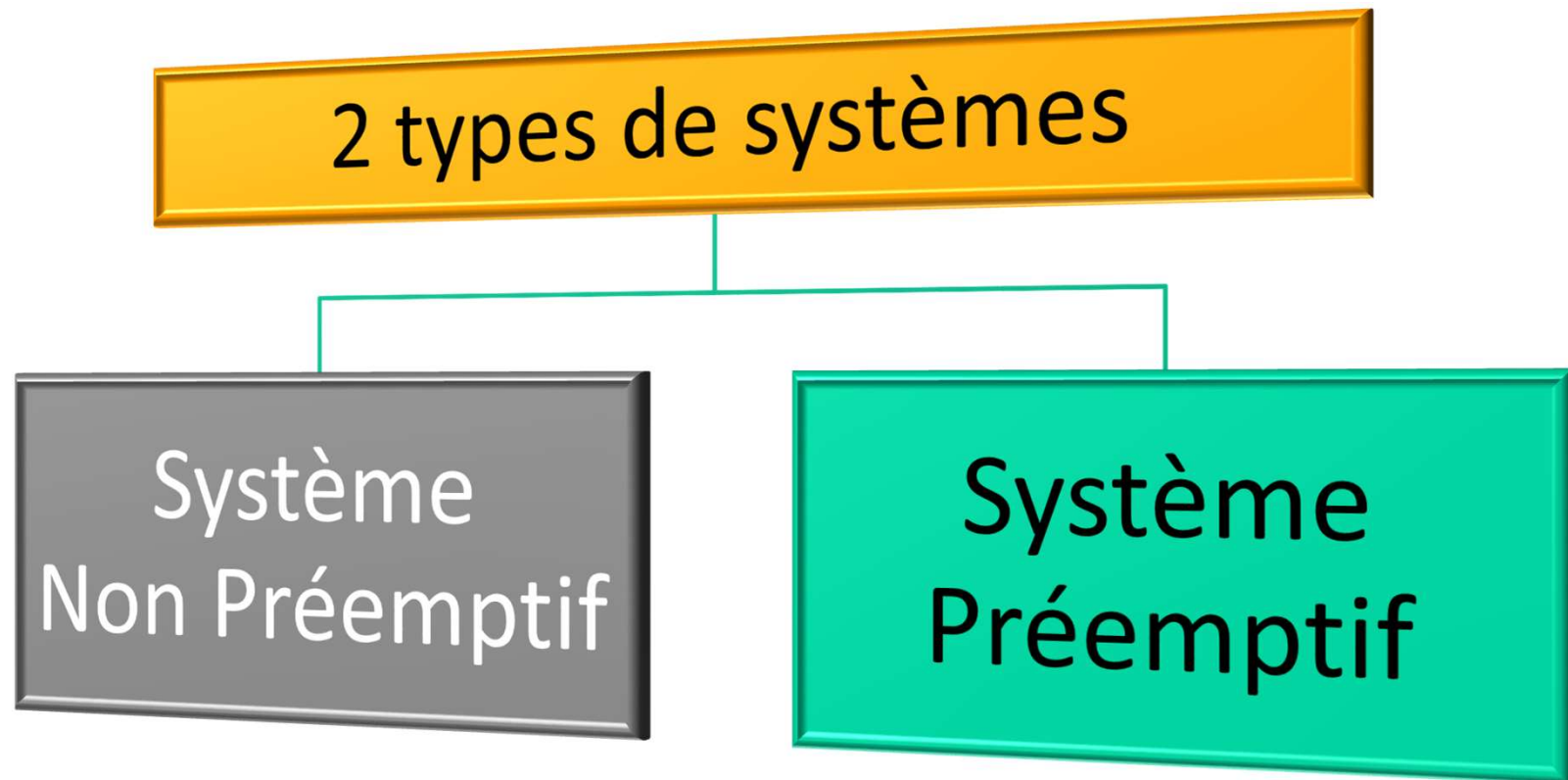
= Ensemble des demandes de ressources à l'instant t

OBJECTIF PRINCIPAL

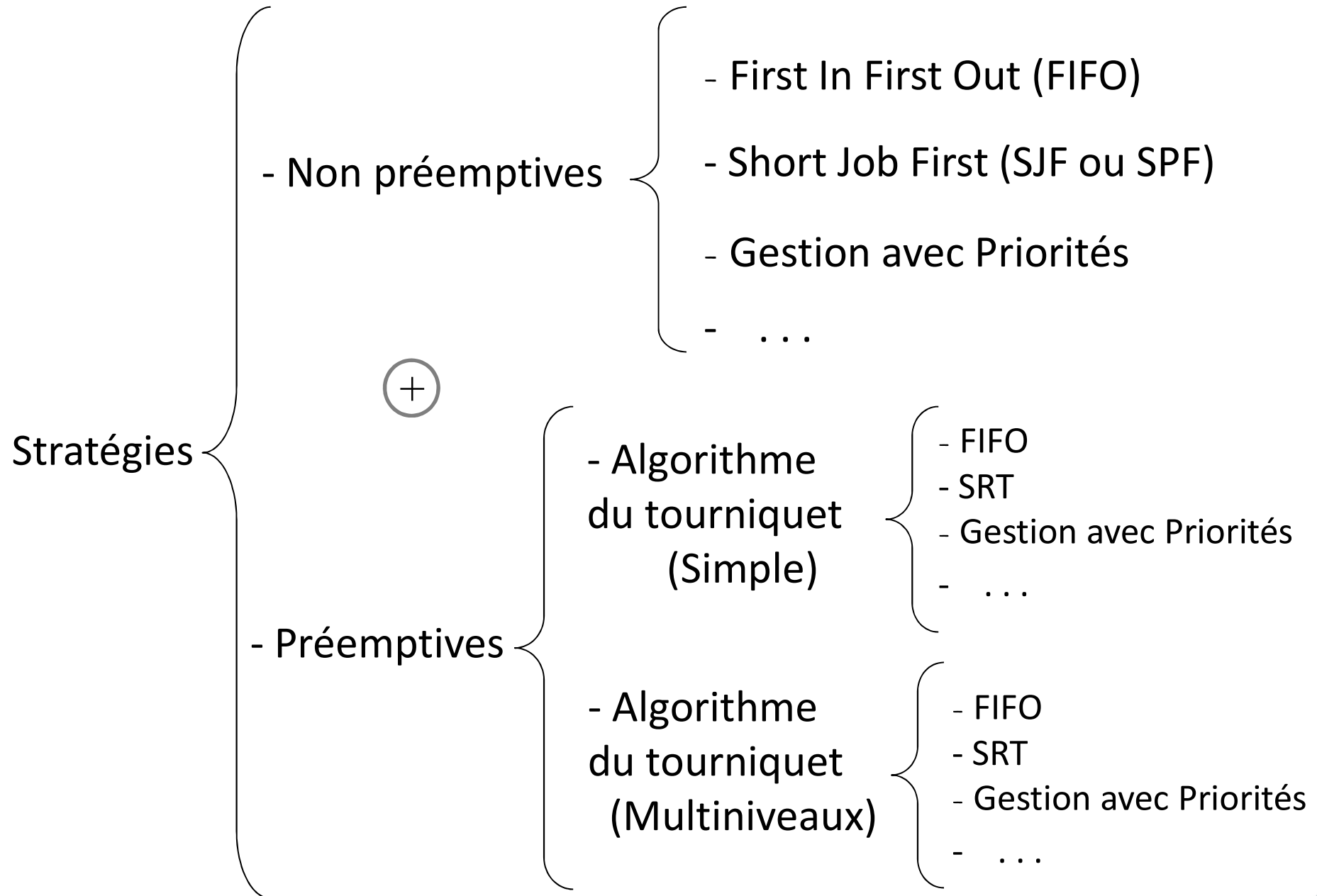
↓ Minimiser la charge du système



2.10 - ORDONNANCEMENT (Gestion d'une File d'Attente)



2.10 – ORDONNANCEMENT: Stratégies de gestion d'une File d'Attente



2.10 – ORDONNANCEMENT : Evaluation des performances



À vous de
Calculer

de = date d'entrée dans la file d'attente

dd = date de début du traitement

df = date de fin de traitement du processus

P0 : de		P1 : de		P0 : dd		P0 : df		P1 : dd			P1 : df				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Chronogramme des évènements

TEMPS MOYEN DE TRAITEMENT

$$\text{Tps Moyen} = \frac{(df1 - de1) + (df2 - de2) + \dots + (dfn - den)}{n}$$

TEMPS MOYEN D'ATTENTE

$$\text{Tps Moyen} = \frac{(dd1 - de1) + (dd2 - de2) + \dots + (ddn - den)}{n}$$

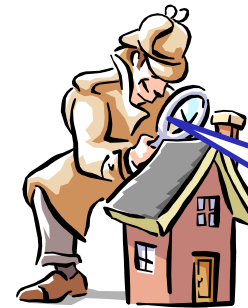


2.11 – ORDONNANCEMENT Non Prémptif : File FIFO



Exemple

	durée	Date Entrée
P1	5	0
P2	4	2
P3	6	3
P4	2	4

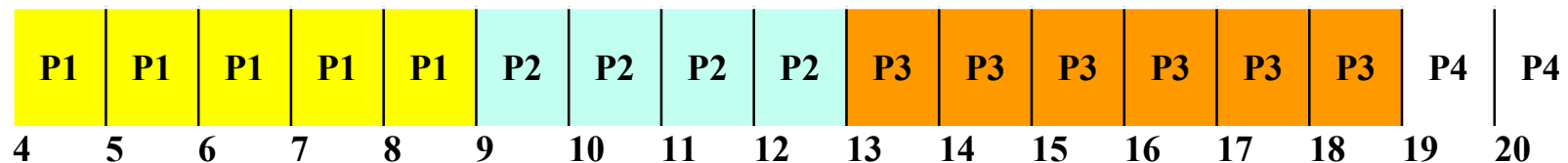


Calculez le
TAM et TTM

Etat de la file d'attente à l'instant t= 4

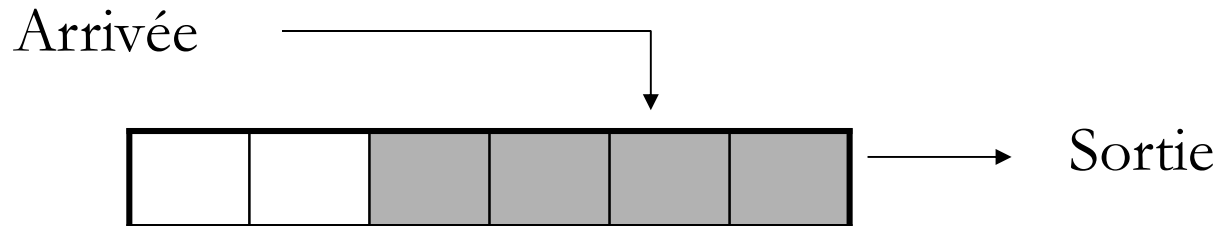


Chronogramme du Processeur



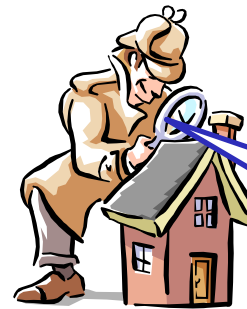
2.12 – ORDONNANCEMENT Non Prémptif : File SJF

SJF : Short Job First Parmi les processus prêts, le processus élu est celui dont la prochaine exécution est la plus courte.



Exemple

	durée	Date entrée
P1	5	0
P2	4	2
P3	6	3
P4	2	4

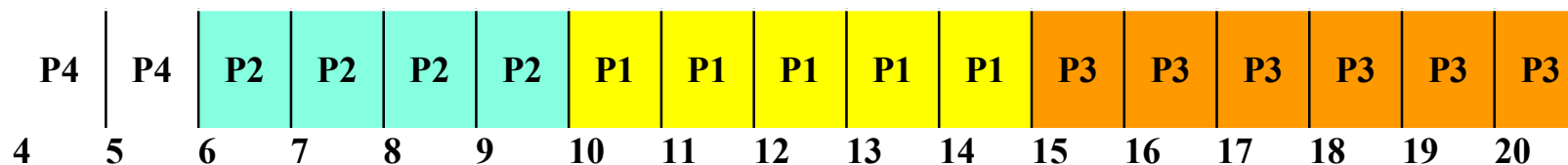


Calculez le
TAM et TTM

Etat de la file d'attente à l'instant $t=4$



Chronogramme du Processeur



2.12 – ORDONNANCEMENT Non Préemptif : File SJF

SJF : Short Job First

Parmi les processus prêts, le processus élu est celui dont la prochaine exécution est la plus courte.

SJF favorise quels types de job ?

Inconvénient majeur du SJF ?



2.12 – ORDONNANCEMENT Non Prémptif : File SJF

La difficulté du SJF est de connaître le temps d'exécution du processus, soit :

- **Le processus annonce la durée de sa prochaine exécution**
- **La durée est estimée en fonction des précédentes exécutions**

Le temps de la prochaine exécution de chaque processus peut être estimé en se basant sur le comportement passé du processus.

Soient S_i et T_i les temps d'exécution estimé et mesuré pour la $i^{\text{ème}}$ rafale, les estimations successives sont :

S_0 ; pour la première

$S_1 = \alpha T_0 + (1 - \alpha) S_0$; pour la seconde

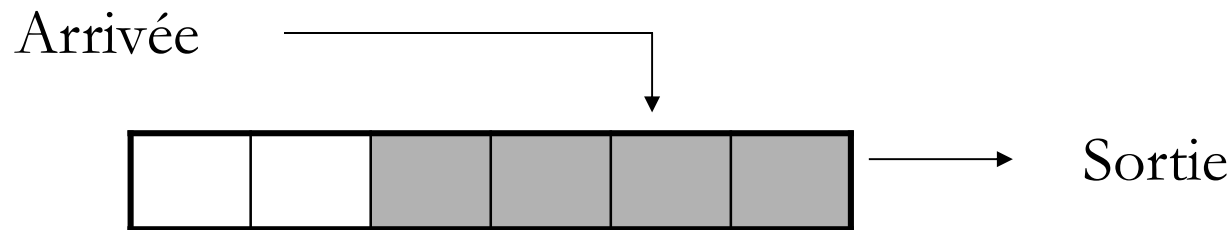
$S_2 = \alpha T_1 + (1 - \alpha) S_1$; pour la troisième

....

$0 \leq \alpha \leq 1$



2.13 – ORDONNANCEMENT Non Préemptif avec Priorité



La priorité peut être :

- annoncée par le processus
- calculée par le système

Exemple de calcul de la priorité :

$$\text{Priorité} = (\text{temps d'attente} + \text{temps d'exécution}) / \text{temps d'exécution}$$



2.13 – ORDONNANCEMENT Non Préemptif avec Priorité

Exemple

L'ordonnanceur est sollicité à l'instant $t = 7$

Calcul de la priorité

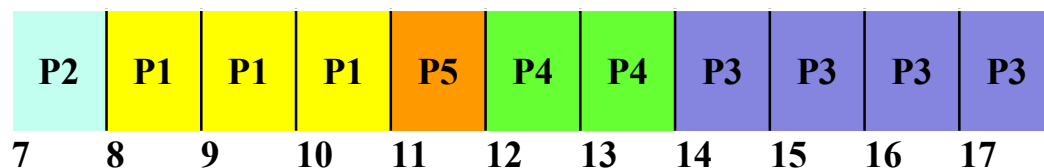
$$\frac{\text{temps d'attente} + \text{temps d'exécution}}{\text{temps d'exécution}}$$

Plus prioritaire = + grande valeur

Processus	Durée	Arrivée	Priorité				
			t=7				
P1	3	0					
P2	1	1					
P3	4	4					
P4	2	6					
P5	1	7					

Calculez le
TAM et TTM

Chronogramme du Processeur



2.14 – ORDONNANCEMENT non Prémptif : ETUDE DE CAS

Processus	Durée	Arrivée
A	5	0
B	1	3
C	4	4
D	2	6
E	2	6

L'ordonnanceur est sollicité à l'instant t=6

Etudiez ce cas avec un
ordonnancement :
FIFO, SJF, et Priorité



Calcul de la priorité

$$\frac{\text{temps d'attente} + \text{temps d'exécution}}{\text{temps d'exécution}}$$

Plus prioritaire = + grande valeur



2.15 – Algorithmes Préemptifs : Concepts

- Pour s'assurer qu'aucun processus ne s'exécute pendant trop de temps, les ordinateurs ont une horloge électronique qui génère périodiquement une interruption.
- A chaque interruption d'horloge, le système d'exploitation reprend la main et décide si le processus courant doit :
 - poursuivre son exécution ou
 - être suspendu pour laisser place à un autre.
- S'il décide de suspendre l'exécution au profit d'un autre, il doit d'abord sauvegarder l'état des registres du processeur avant de charger dans les registres les données du processus à lancer (commutation de contexte).
- Cette sauvegarde est nécessaire pour pouvoir poursuivre ultérieurement l'exécution du processus suspendu.



Nécessité que le système soit doté d'une horloge

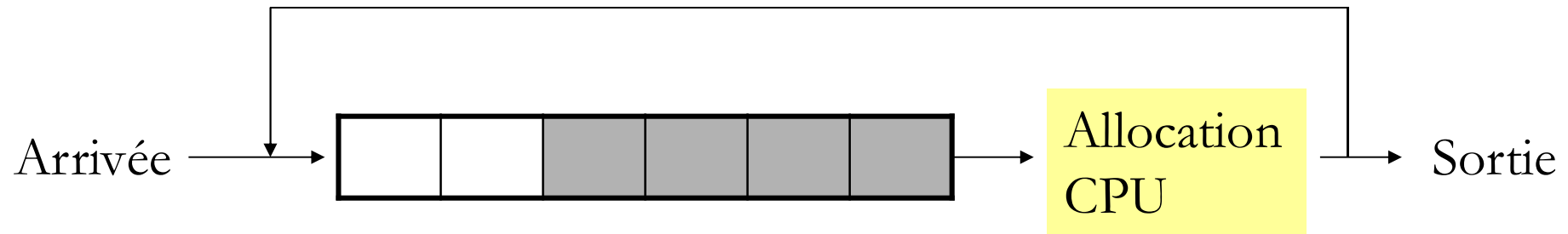


2.15 – Algorithmes Préemptifs : Concepts

- Le processeur passe donc d'un processus à un autre en exécutant chaque processus pendant quelques dizaines ou centaines de millisecondes.
- Le temps d'allocation du processeur au processus est appelé **quantum**.
- La **commutation de contexte** entre processus doit être rapide (temps nettement inférieur au quantum).
- Le processeur, à un instant donné, n'exécute réellement qu'un seul processus.
- Pendant une seconde, le processeur peut exécuter plusieurs processus et donne ainsi l'impression de parallélisme (pseudo-parallélisme).



2.16 – Algorithmes Préemptifs : Tourniquet (ROUND ROBIN)



👉 **choix de la durée du quantum**

- Il était de 1 seconde dans les premières versions d'UNIX.
- Il varie entre 20 et 120 ms.



SRT : Plus court temps restant

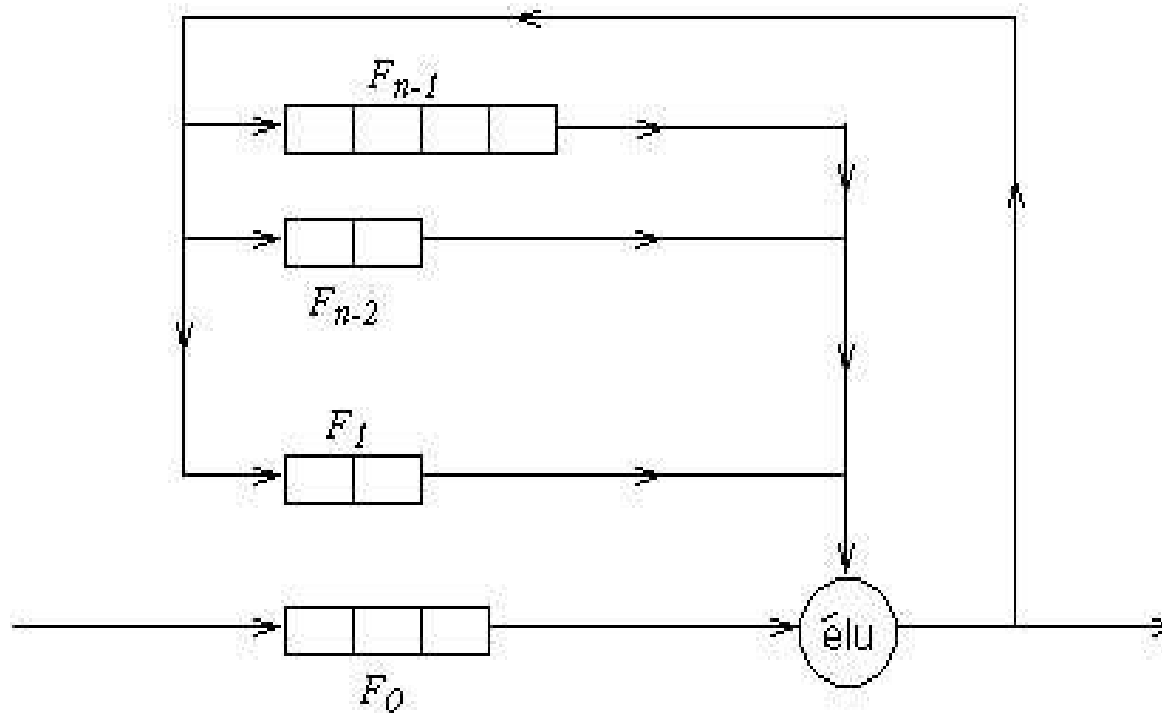
Gestion avec Priorités

Loterie



2.18 – Algorithmes Préemptifs : Tourniquet Multi-niveaux

Stratégie fondée sur la priorité

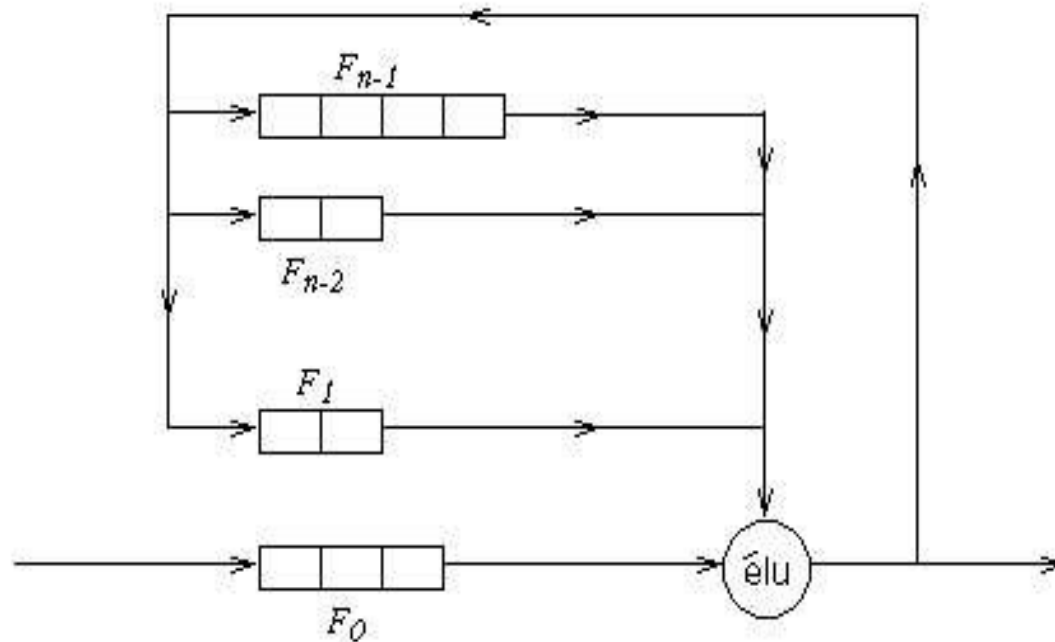


2.18 – Algorithmes Préemptifs : Tourniquet Multi-niveaux

Les Options possibles du Tourniquet Multi-niveaux

- Niveau
(Nombre)

- Le Mode de gestion de la File d'attente
- La priorité
- Le quantum



*Toutes ces options permettent
d'être cohérent avec l'objectif
de l'ordonnanceur*



2.19 – ORDONNANCEMENT : Conclusion

Quelques critères de bon ordonnancement :

Le temps d'attente : temps entre la soumission du processus et le début de son traitement

Le temps de traitement : temps entre la soumission du processus et son achèvement

Débit : nombre de processus terminés / unité de temps

Charge du Système

Taux d'utilisation de l'unité centrale



2.20 – ORDONNANCEMENT des Threads

Implémentation dans
l'espace utilisateur

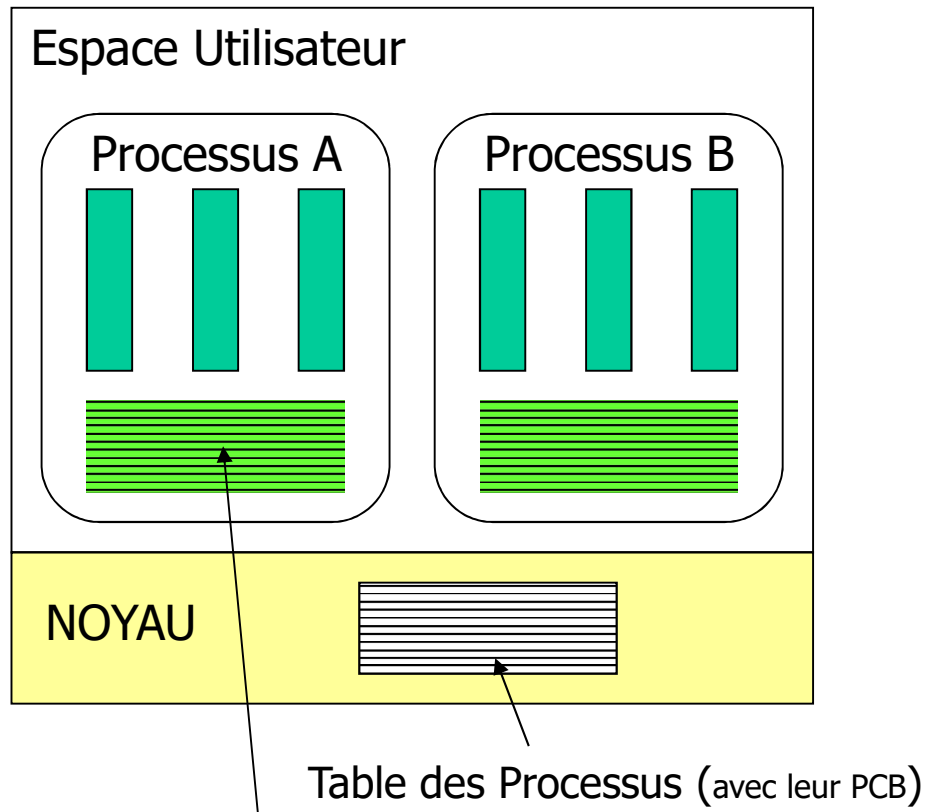
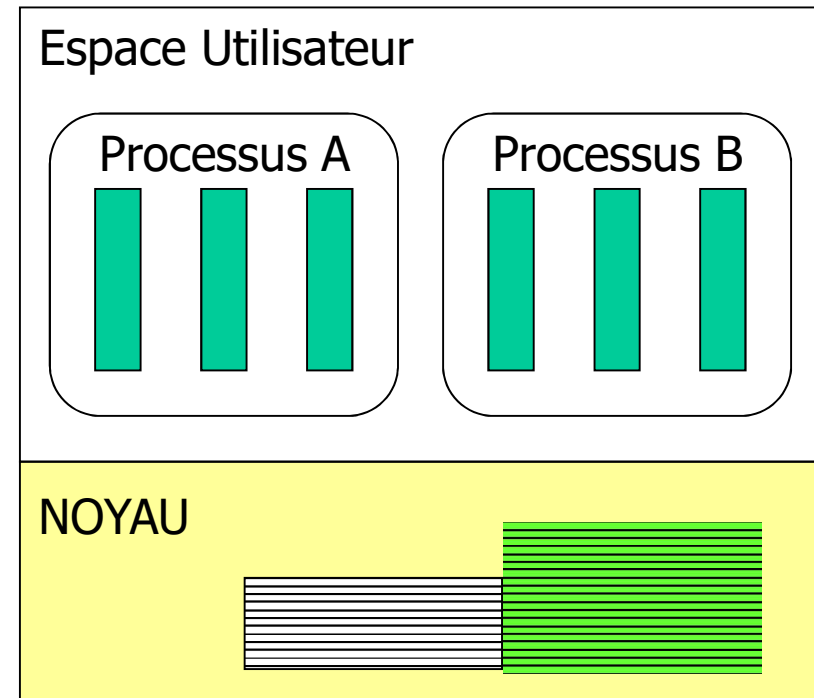


Table des Threads

Implémentation dans le
Noyau



Thread

Complément de cours : Les Processus légers



2.21 – ORDONNANCEMENT : UNIX

- C'est un ordonnanceur à deux niveaux. L'ordonnanceur de bas niveau se charge d'élire un processus parmi ceux qui résident en mémoire.
- L'ordonnanceur de haut niveau se charge des transferts de processus entre la mémoire centrale et le disque.
- L'ordonnanceur de bas niveau utilise plusieurs files, une priorité est associée à chaque file (plusieurs niveaux de priorité).
- Les processus prêts qui sont en mémoire, sont répartis dans les files selon leur priorité.
- Les priorités des processus s'exécutant en mode utilisateur sont positives ou nulles, alors que celles des processus s'exécutant en mode noyau sont négatives.
- Les priorités négatives sont les plus élevées.



2.21 – ORDONNANCEMENT : UNIX

L'ordonnanceur de bas niveau choisit le processus le plus prioritaire qui se trouve en tête de file.

- Le processus élu est alors exécuté pendant au maximum un quantum (100 ms). S'il ne se termine pas ou ne se bloque pas au bout de ce quantum, il est suspendu.
- Le processus suspendu est inséré en queue de sa file.
- La priorité initiale d'un processus utilisateur est en général égale à 0. La commande nice permet d'attribuer une priorité plus basse à un processus.
- Les priorités des processus prêts en mémoire sont recalculées périodiquement toutes les secondes.



2.21 – ORDONNANCEMENT : LINUX

L'ordonnanceur de Linux est un ordonnanceur préemptifs et à priorité.

- Il distingue trois politiques d'ordonnancement :
 - **SCHED_FIFO : FIFO** : Aucun processus de cette classe ne peut être interrompu par un autre, à moins que ce soit un processus plus prioritaire de la même classe.
 - **SCHED_RR** (Tourniquet)
 - **SCHED_OTHER** (les moins prioritaires) : Ils s'exécutent uniquement si aucun processus des classes 1 et 2 n'est prêt.
- Chaque processus créé fait partie d'une des trois classes et a une priorité comprise entre 1 et 40 (20 par défaut).
- Les valeurs élevées correspondent à des priorités élevées.

Les processus sous les politiques SCHED_FIFO et SCHED_RR sont considérés comme des processus « temps réels » et sont donc plus prioritaires par rapport aux processus qui sont sous la politique par défaut SCHED_OTHER.



2.21 – ORDONNANCEMENT : LINUX

Chaque processus de la deuxième et troisième classe a un Quantum :
valeur initiale = priorité.

- Le Quantum du processus en cours d'exécution diminue de 1 unité à chaque consommation de 10 ms de temps CPU.
- Les quanta sont réajustés lorsque tous les processus sont bloqués ou leurs quanta sont nuls : $\text{Quantum} = \text{Quantum} / 2 + \text{priorité}$
- Lorsque l'ordonnanceur prend le contrôle, il attribue une note à chaque processus
- Le processus élu est celui qui a une plus forte note.



2.22 – ORDONNANCEMENT : WINDOWS (*Classes de thread*)

Windows XP ne contient pas un module autonome qui se charge de l'ordonnancement. Le code est dispersé.

- Il ordonnance les threads sans se préoccuper de leurs processus.
- C'est un ordonnanceur **préemptif à priorité** (32 niveaux de priorité). Les priorités sont dynamiques.
- On distingue quatre classes de processus légers :
 - **Les threads temps réel (Time Critical)** : chaque thread de cette classe à une priorité comprise entre 16 et 31. Un thread de cette classe requiert l'utilisation totale du processeur. Cette classe est réservée à des utilisations très particulières (ex : un gestionnaire de souris).
 - **Les threads utilisateurs (High)** : chaque thread de cette classe a une priorité comprise entre 1 et 15. Sa priorité varie durant sa vie mais reste toujours comprise entre 1 et 15. Il migre d'une file à une autre avec l'évolution de sa priorité. Cette classe ne doit généralement être accordée qu'à des applications effectuant des tâches très brèves ou étant en premier plan



2.22 – ORDONNANCEMENT : WINDOWS (*Classes de thread*)

– **Le thread zéro (Normal)** : il s'exécute, en arrière plan, lorsque toutes les files, de priorité supérieure, sont vides.

Un thread de cette classe se verra allouer le processeur exactement autant de fois que les autres threads de la même priorité.

Si deux threads utilisent cette priorité et utilisent le processeur en permanence, les performances de chacun des thread seront divisées par 2. La quasi totalité des threads utilise cette priorité (c'est d'ailleurs la plus recommandée hors cas très particuliers).

– **Le thread idle** : un thread de cette classe se verra allouer le processeur seulement si aucune autre application ne demande le contrôle.

De manière générale il est inutile de modifier la priorité des threads. La modification des priorités ne doit être faite que dans les cas particuliers et en connaissance de cause. En particulier, les priorités 'Time Critical' et 'High' ne doivent jamais être utilisées abusivement.



2.22 – ORDONNANCEMENT : WINDOWS (*Astuces*)

Quelques astuces de gestion de Windows

- Les processus légers de même priorité sont dans une même file et ordonnancés selon l'algorithme du tourniquet.
- L'ordonnanceur est appelé si :
 - Un thread passe à l'état prêt ou quitte l'état d'exécution.
 - La priorité d'un thread change.
- L'ordonnanceur choisit le processus léger le plus prioritaire en tête de file et lui alloue le processeur pendant au maximum un quantum.
- Quand un processus léger est sélectionné, son exécution peut être interrompue (avant la fin du quantum) si un autre processus léger de priorité plus élevée devient prêt. Le thread préempté est remis en tête de la file associée à son niveau de priorité.
- Si un thread est suspendu car il a consommé son quantum, sa priorité peut être diminuée.



2.22 – ORDONNANCEMENT : WINDOWS (*Astuces*)

- Si un thread est interrompu pour une requête d'E/S, un sémaphore, etc., verra son quantum diminuer et par contre sa priorité augmenter. Elle augmente encore plus, s'il s'agit d'une requête d'E/S qui concerne un clavier ou un écran.
- Si un thread à l'état prêt reste trop longtemps en attente du processeur (depuis 4 secondes environ), il se voit attribuer une priorité 15 et un quantum de 4 unités (accélérer le threads pour éviter le problème de famine).
- À la fin du quantum, le thread reprend sa priorité de base originale.
- C'est le « balance set manager » qui se charge de la recherche de threads en famine (toutes les secondes).
- Afin de minimiser le temps CPU consommé par le balance set manager, au maximum 16 threads sont examinés à chaque passage. S'il y en a plus, il examinera les 16 suivants à son prochain passage.
- Le balance set manager limite le nombre de threads à accélérer à 10.000



2.23 – ORDONNANCEMENT suite et FIN

Ordonnancement des systèmes Multiprocesseurs

- L'ordonnanceur doit choisir à la fois le processus à exécuter et le processeur qui le prendra en charge.

Ordonnancement des systèmes Multi-Ordinateurs

- c'est une autre histoire !!!



3 – Les Risques de la Multiprogrammation : La Concurrency

Des processus concurrents peuvent être des processus **coopératifs** ou **indépendants**.

- Un processus est **indépendant** *s'il n'affecte pas les autres processus ou ne peut pas être affecté par eux.*
- Un processus qui ne partage pas de données avec d'autres processus est indépendant.
- Un processus est **coopératif** *s'il peut affecter les autres processus en cours d'exécution ou être affecté par eux.*
- Un processus qui partage des données avec d'autres processus est un processus coopératif.



3 – Les Risques de la Multiprogrammation : La Concurrency

- Les données partagées par les processus coopératifs peuvent se trouver en mémoire principale ou en mémoire auxiliaire (dans un fichier)
- Les accès concurrents à des données, des ressources partagées peuvent produire :

❑ - des **inter-blocages** (*si l'accès est « limité »*)

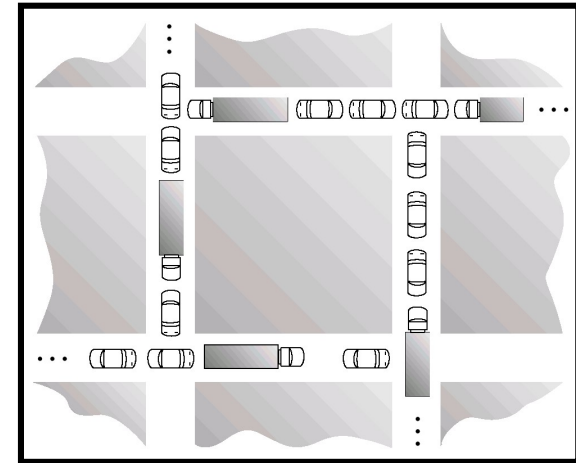
❑ - des **incohérences de données, de résultats**
(*si l'accès n'est pas « limité »*)



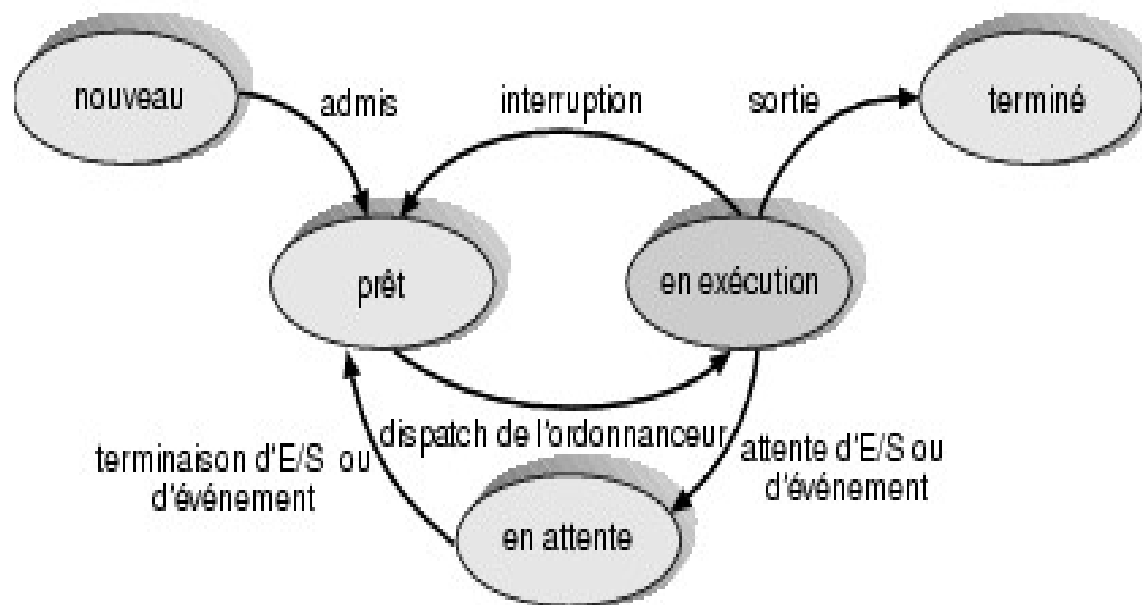
3.1 – Les Risques de la Multiprogrammation : INTERBLOCAGE

Définition

Un ensemble de processus est en *inter blocage* si chaque processus attend un événement que seul un autre processus de l'ensemble peut provoquer. *Mais comme il est bloqué il ne pourra jamais le provoquer.*



Un processus en inter blocage est en état **bloqué (attente)**



3.1 – INTERBLOCAGES : Exemples

2 Cuisiniers, 1 verre doseur, 1 fait-tout

1^{ère} recette exécutée par le cuisinier A:

- 1- Mettre du beurre dans le fait-tout,
- 2 - Mesurer la farine dans le verre doseur,
- 3 - Mélanger dans le fait-tout

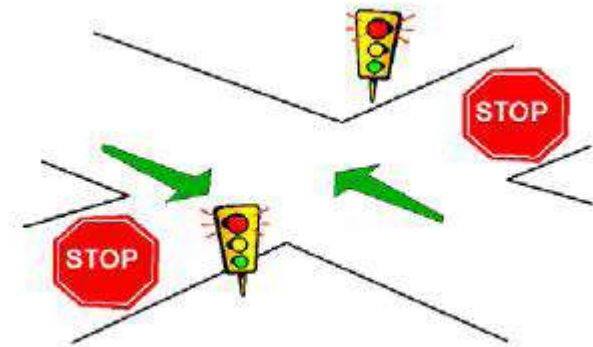
2^{ième} recette exécutée par le cuisinier B:

- 1- Mesurer de l'eau dans le verres doseur
- 2 - Mettre des œufs dans le fait- tout,
- 3 - puis mélanger l'eau.

Exemple des feux rouges

Risque d'accident lorsque tous les feux sont verts

On dit que les feux sont en exclusion mutuelle.



3.1 – INTERBLOCAGES : Les conditions

L'interblocage demande la présence simultanée de 4 conditions
(conditions nécessaires et suffisantes)

1. **La ressource est à accès limitée** (voir en exclusion mutuelle)
2. **Saisie et attente** : un processus a saisi une ressource non partageable et en attend d'autres pour continuer sa tâche
3. **Pas de préemption**: un processus qui a saisi une ressource non partageable la garde jusqu'à ce qu'il aura terminé sa tâche
4. **Attente circulaire**: il y a un cycle de processus tel que chaque processus pour compléter doit utiliser une ressource non partageable qui est utilisée par le suivant, et que le suivant gardera jusqu'à sa terminaison
 - **En présence des 3 premières conditions, une attente circulaire est un interblocage**
 - **Les 3 premières conditions n'impliquent pas nécessairement interblocage, car l'attente circulaire pourrait ne pas se vérifier**



3.2 – INTERBLOCAGES : Gestion des Inter-blocages

Empêcher les
inter-blocages

- **Stratégie de Prévention**
- **Stratégie d'Évitement**

Corriger les
inter-blocages

- **Stratégie de détection-guérison**

**Ignorer les
inter-blocages**

- **Stratégie de l'Autruche**



3.3 – INTERBLOCAGES: **Stratégie de Prévention**

-S'attaquer à la condition de détention et d'attente :

forcer les processus à demander toutes leurs ressources avant l'exécution, ou bien une seule à la fois.

Inefficace et risque de famine.

- Ordonner les ressources et ne permettre de les demander que dans un ordre croissant

- Pas de Requisition

si un processus requiert une ressource occupée, toutes ses ressources sont implicitement libérées et le processus passe en attente.

Pas utilisable pour certains types de ressources (imprimantes. . .).



3.3 – INTERBLOCAGES : Stratégie d'évitement

Notion d'Etat Sûr

*Un état est dit sûr s'il existe une suite d'états ultérieurs qui permette à **tous les processus** d'obtenir toutes leurs ressources et **de se terminer**.*

Un état sûr peut être défini ainsi : il existe un moyen d'ordonnancer les processus sans qu'il y ait de risque de blocage.

Algorithme du Banquier

Algorithme mis au point par Edgar Dijkstra en 1965 pour éviter les problèmes d'interblocages et gérer l'allocation des ressources.

Cet algorithme est nommé ainsi car il reproduit le modèle du prêt à des clients par un banquier.



3.5 – INTERBLOCAGES : Stratégie détection-guérison

Détection

Enoncé

- Le processus 1 détient la ressource A et demande B
- Le processus 2 demande la ressource C
- Le processus 3 demande la ressource B
- Le processus 4 détient D et demande B et C
- Le processus 5 détient C et demande E
- Le processus 6 détient F et demande B
- Le processus 7 détient E et demande D

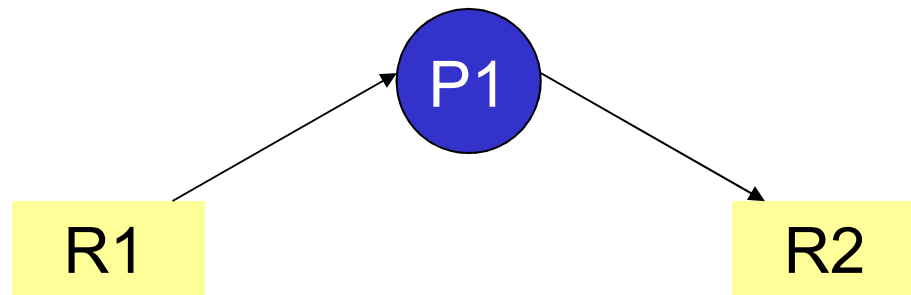
Y a t-il situation
d'interblocage ?



3.5 – INTERBLOCAGES : Représentation

On modélise l'utilisation des ressources à l'aide d'un graphe

- Un ensemble de sommets V et d'arêtes E
- V est partitionné dans:
 - $P = \{P_1, P_2, \dots, P_n\}$, l'ensemble de tous les processus dans le système
 - $R = \{R_1, R_2, \dots, R_m\}$, l'ensemble de tous les types de ressources dans le système
- arête requête – arête dirigée $P_i \rightarrow R_k$
- arête affectation – arête dirigée $R_i \rightarrow P_k$



- Deux types de nœuds :

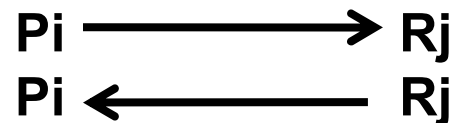
– processus P_i

– ressources R_j

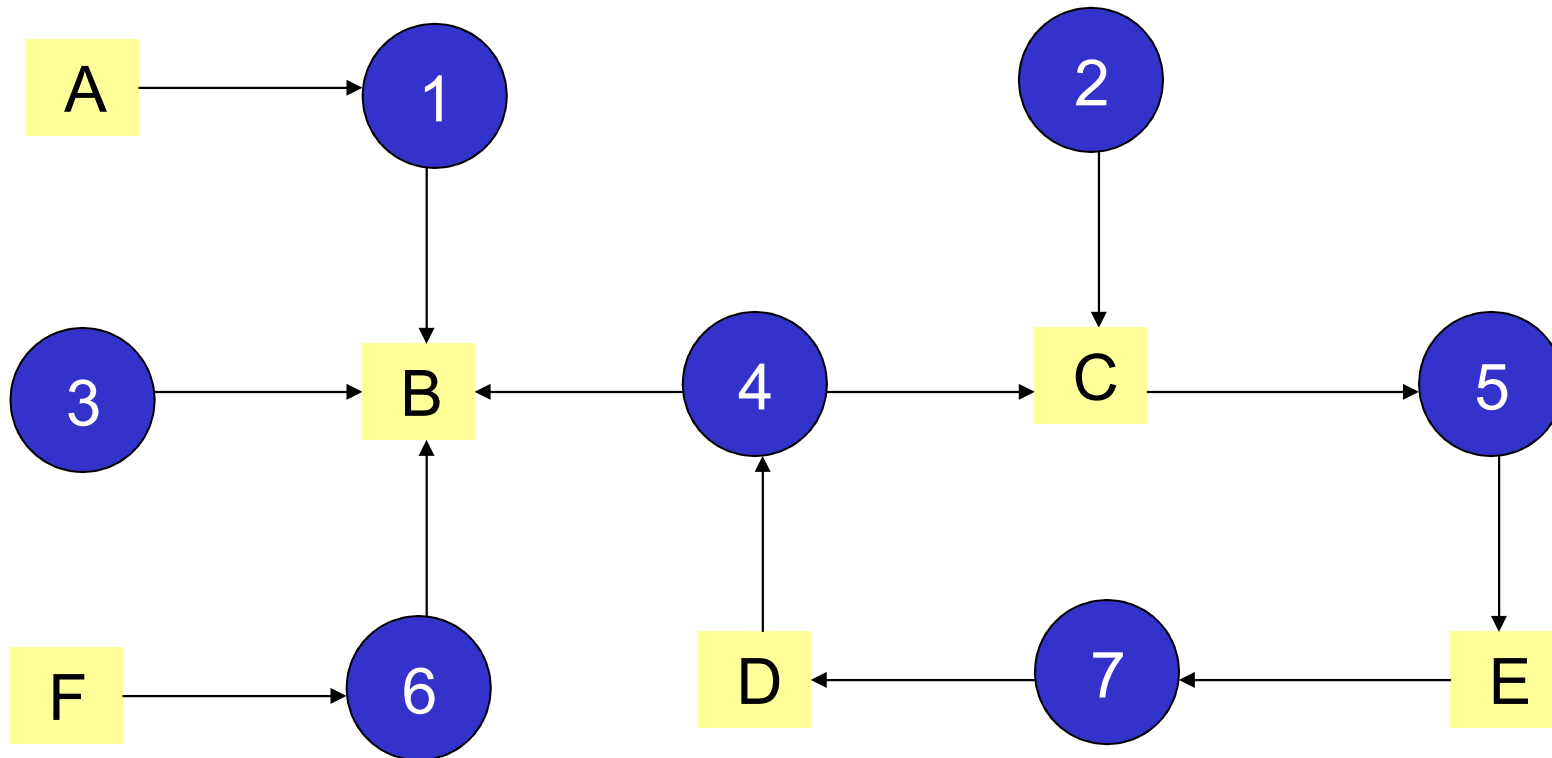
- Deux types d'arcs :

– Arcs de requête

– Arcs d'affectation



3.6 – INTERBLOCAGES : Stratégie détection-guérison



3.7 – INTERBLOCAGES: **Stratégie détection-guérison**

Détection

A quel moment doit on essayer de détecter un éventuel interblocage ?

-
-
-

Guérison

Que proposer pour guérir ?

- Reprise par Rollback

- Reprise par Suppression de Processus



3.7 – INTERBLOCAGES: Stratégie détection-guérison

Terminaison de Processus

Pas simple !!!!

- **Terminaison totale :**
coût élevé pour l'utilisateur (les processus avortés devront être relancés).
- **Terminaison partielle :**
un par un jusqu'à ce que le cycle d'interblocage soit rompu.
Coût élevé pour le système (appels multiples à l'algorithme de détection).
- **Terminaison externe :**
tuer un processus non interbloqué (parce qu'il occupe des ressources intéressantes).

Critères de choix pour la terminaison :

- ☐ priorité du processus,
- ☐ âge du processus, temps restant d'exécution,
- ☐ nombre et type de ressources utilisées,
- ☐ ...



3.8 – INTERBLOCAGES : Stratégie d'évitement

Algorithme du Banquier : Principes

- Soient 5 processus (P1 P2 P3 P4 P5) utilisant 4 ressources
- Soit A la matrice des ressources actuellement allouées au processus
- Soit S la matrice des ressources restantes à allouer à chacun des processus

A	P1	3	0	1	1
	P2	0	1	0	0
	P3	1	1	1	0
	P4	1	1	0	1
	P5	0	0	0	0
	P	5	3	2	2
	E	6	3	4	2
	R	1	0	2	0

S	P1	1	1	0	0
	P2	0	1	1	2
	P3	3	1	0	0
	P4	0	0	1	0
	P5	2	2	2	0

P : Vecteur des nombres de ressources allouées de chaque type

E : Vecteur des nombres de ressources de chaque type existantes dans le système

R : Vecteur des nombres de ressources restantes de chaque type

- L'ordonnanceur devra choisir un processus dont son vecteur dans la matrice S est inférieur au vecteur R.



3.8 – INTERBLOCAGES : Exercice

Un système contient 4 processus et 5 ressources.

	Attribuées	Maximum
P1	10211	11212
P2	20110	22210
P3	11010	21310
P4	11110	11221

Disponibles à l'instant t
0 0 x 1 1



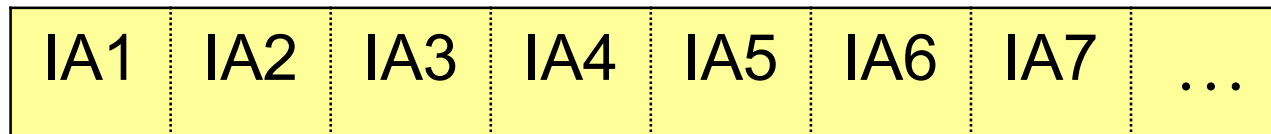
Quelle est la plus petite
valeur de x pour laquelle
l'état est sur ?



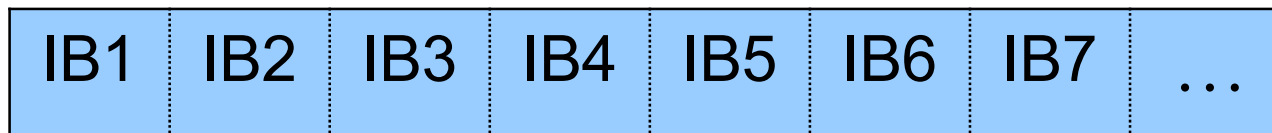
3.9 – Les Risques de la Multiprogrammation : Ordre d'exécution

L'ordre d'exécution des instructions de différents processus est aléatoire !!!

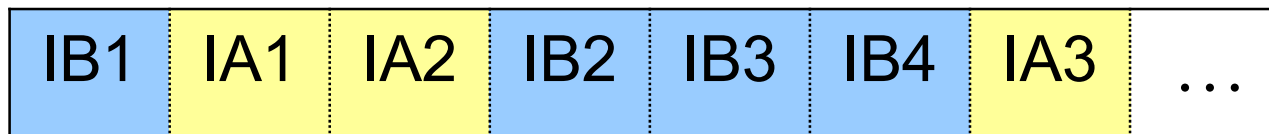
processus A



processus B



Ordre d'Exécution possible



3.9 - Les Risques de la Multiprogrammation : Etude de cas

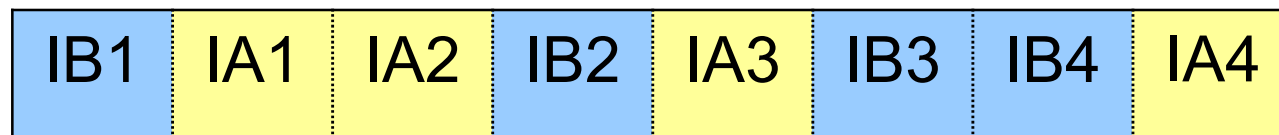
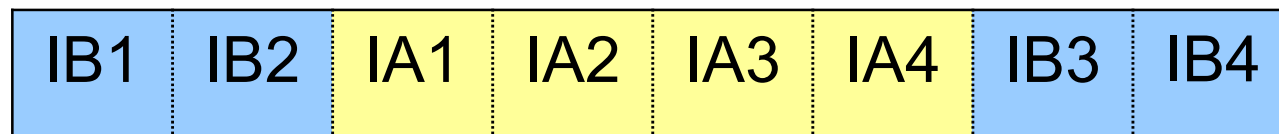
Considérons une ressource : ici une variable : `int cpt=2000`

processus A

IA1	charger(varA2,1000)
IA2	charger(varA1,cpt)
IA3	resA=add(varA1,varA2)
IA4	ranger(resA,cpt)

processus B

IB1	charger(varB2,2000)
IB2	charger(varB1,cpt)
IB3	resB=sub(varB1,varB2)
IB4	ranger(resB,cpt)



3.10 – SYNCHRONISATION DES PROCESSUS : Concurrency

Pour éviter les problèmes d'incohérence sur les résultats :

- **les processus doivent se synchroniser**

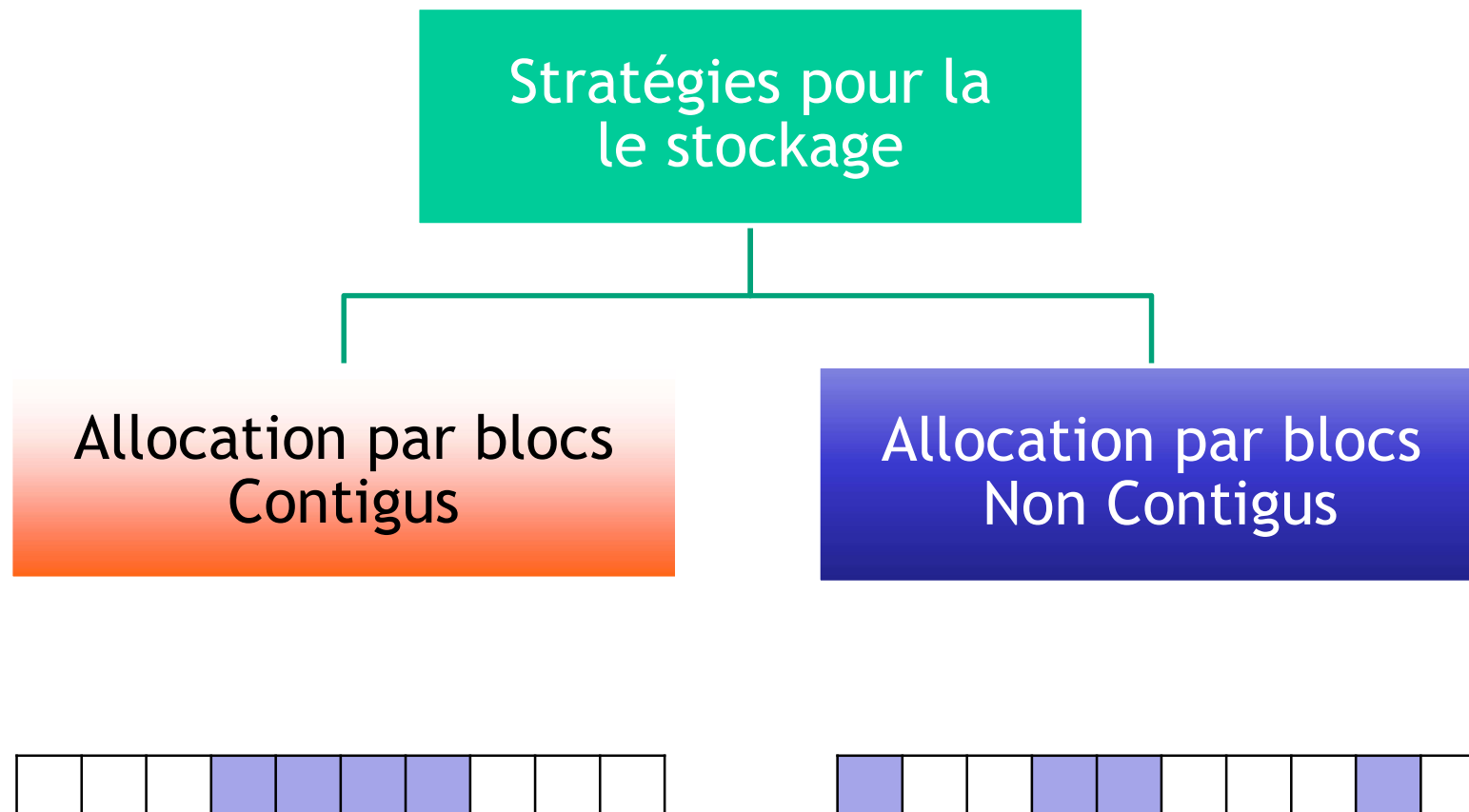
Mais la synchronisation peut entraîner le blocage d'un voir plusieurs processus désirant accéder à « une donnée sensible » utilisée par un autre processus.

- Et de ce fait **la synchronisation peut entraîner des risques d'interblocage.**

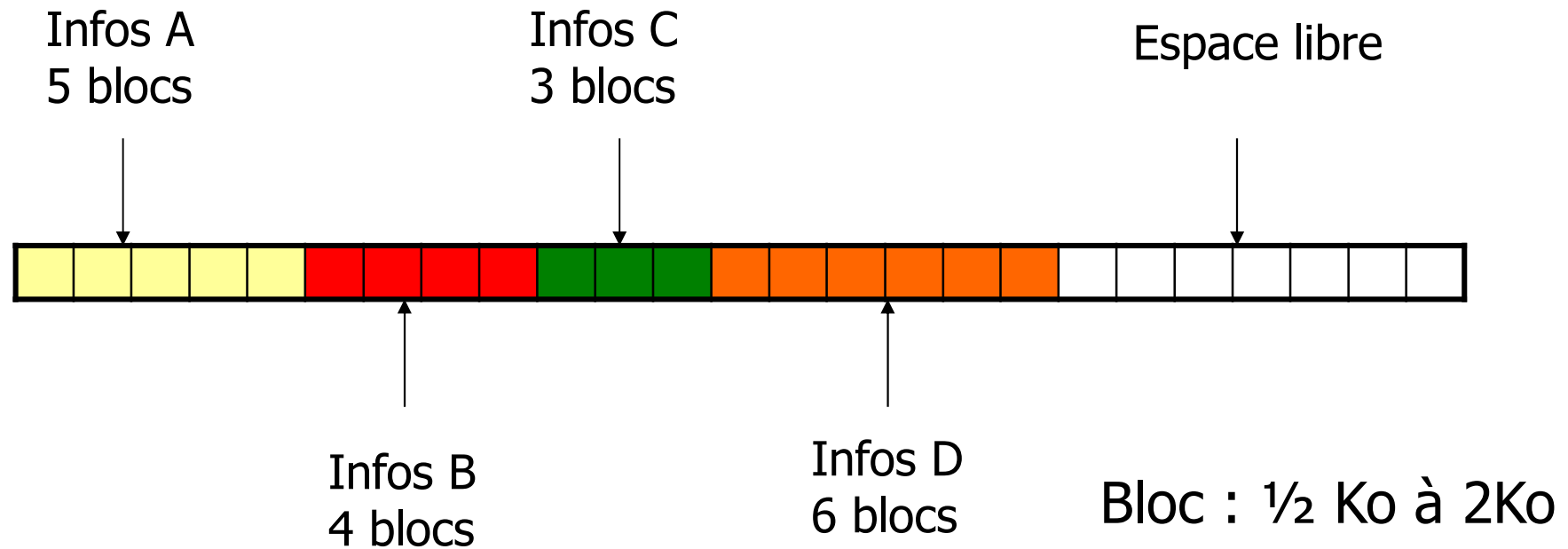
*Ces risques d'interblocages peuvent être évités en utilisant une **bonne stratégie de synchronisation.***



4 – Gestion de Mémoire : 2 stratégies pour le stockage



4.1 – Mémoires : Allocation par blocs contigus

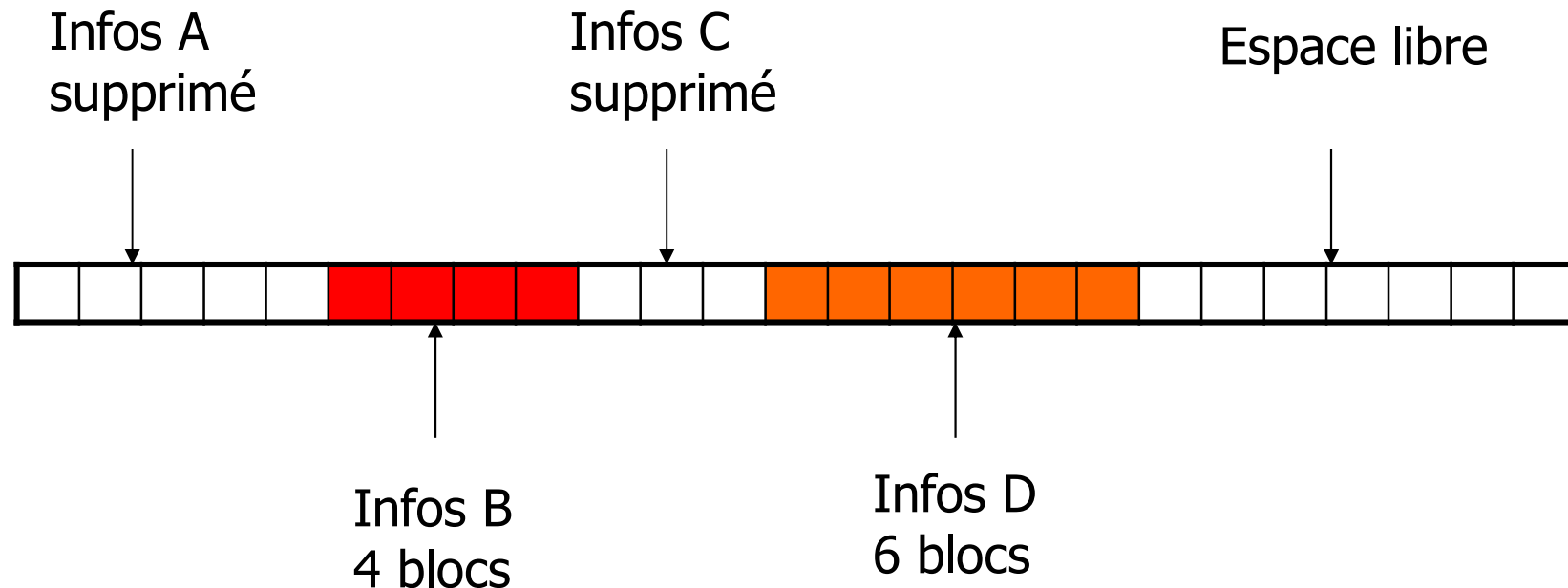


L'allocation d'une Info requiert de trouver un espace libre dans la mémoire suffisamment grand.



4.1 – Mémoires : Allocation par blocs contigus

Après suppression des Infos A et C



Une demande d'allocation d'une Info de taille 8 blocs n'est pas réalisable malgré un espace libre de 15 blocs.

Fragmentation de l'espace libre de la mémoire

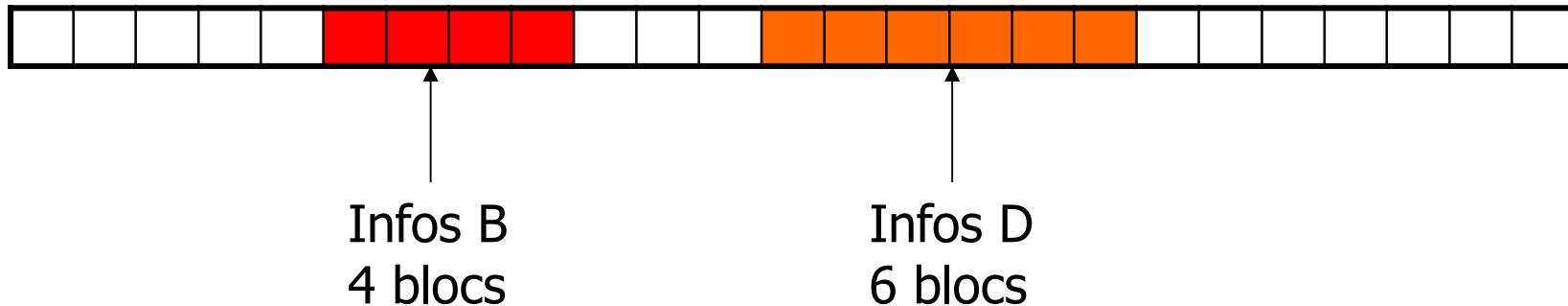
-- > solution : défragmentation des l'espace libre



4.1 – Mémoires : Allocation par blocs contigus

3 méthodes principales permettent de choisir l'espace d'allocation :

- First-Fit : le premier espace pouvant répondre à la demande
- Best-Fit : l'espace qui donne le plus petit résidu
- Worst-Fit : l'espace qui donne plus grand résidu

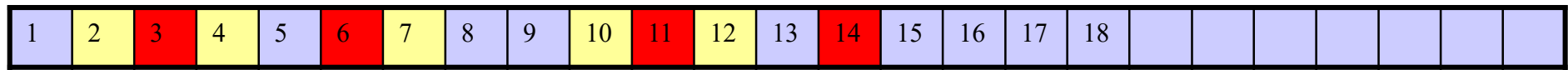


3 demandes d'allocation arrivent dans l'ordre suivant :

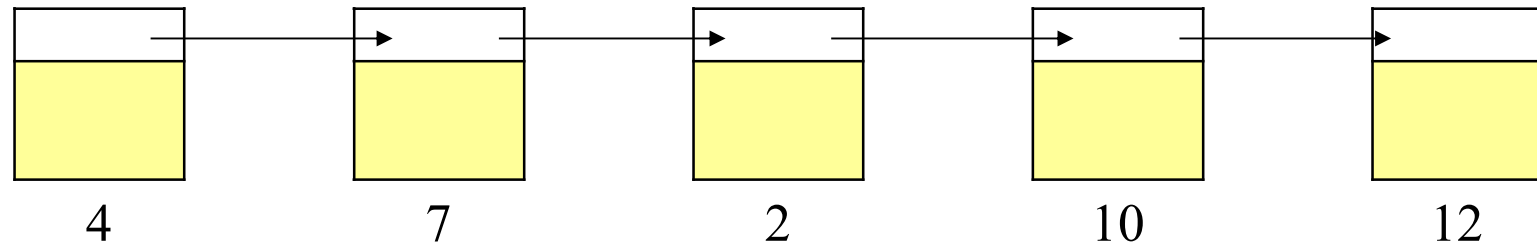
- Info F de 3 blocs
- Info G de 5 blocs
- Info H de 4 blocs



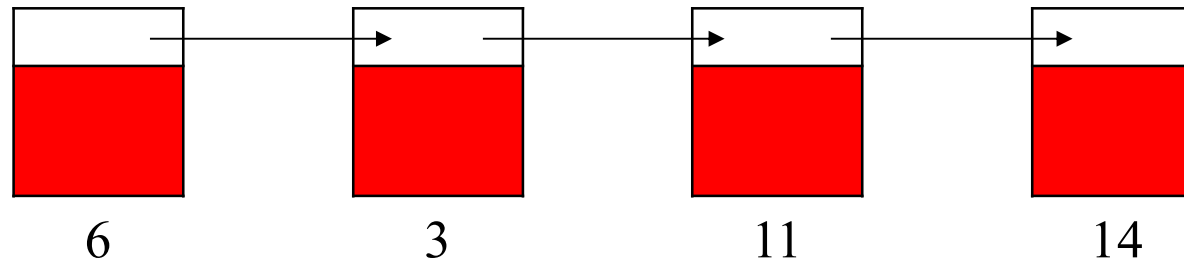
4.2 – Mémoires : Allocation par blocs non contigus chaînés



Infos A



Infos B



Allocation par blocs non contigus et liste chaînée



4 .3 – Mémoires : Allocation par liste chaînés avec table

On range le pointeur de chaque bloc dans une table

1	
2	10
3	11
4	7
5	
6	3
7	2
8	
9	
10	12
11	14
12	-1
13	
14	-1
15	

Infos A : blocs 4,7,2,10 et 12

Infos B : blocs 6,3,11 et 14

+ le bloc est utilisé que pour les données pas besoin de stocker en entête de bloc le pointeur

+ la table étant chargé en mémoire l'accès aléatoire est facilité sans accès disque supplémentaire

+ Il suffit de stocker pour chaque Info le bloc de départ.



4.4 – Mémoires : Allocation indexée

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18								
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	--	--	--	--	--	--	--	--

L'allocation indexée vise à supprimer les deux inconvénients du mode d'allocation par blocs chaînés

Constitution d'une table d'index pour chaque Infos.

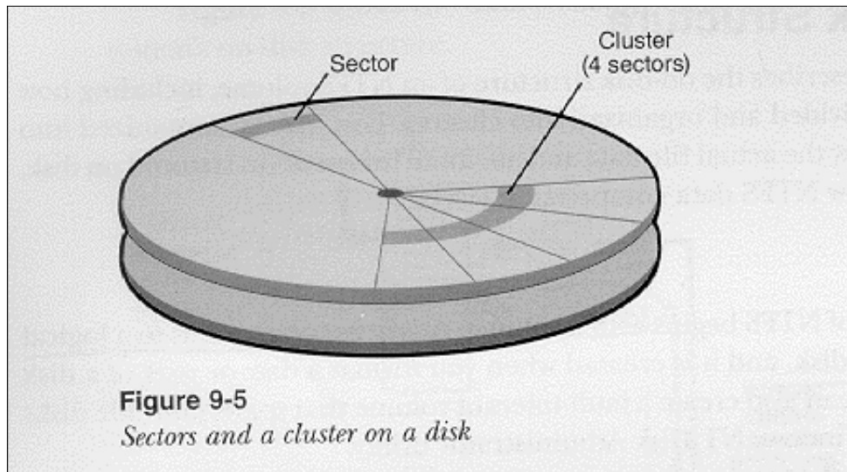
Index Info A
4
7
2
10
12

Index Info B
6
3
11
14



5.1 – Mémoires Disques : SGF - FAT 16

- Ils utilisent une **table d'allocation de fichiers**



FAT16 est un système 16 bits
 2^{16} , soit 65536 blocs (ou clusters)

la taille maximale d'une partition FAT se trouve en multipliant le nombre de clusters par la taille d'un cluster.

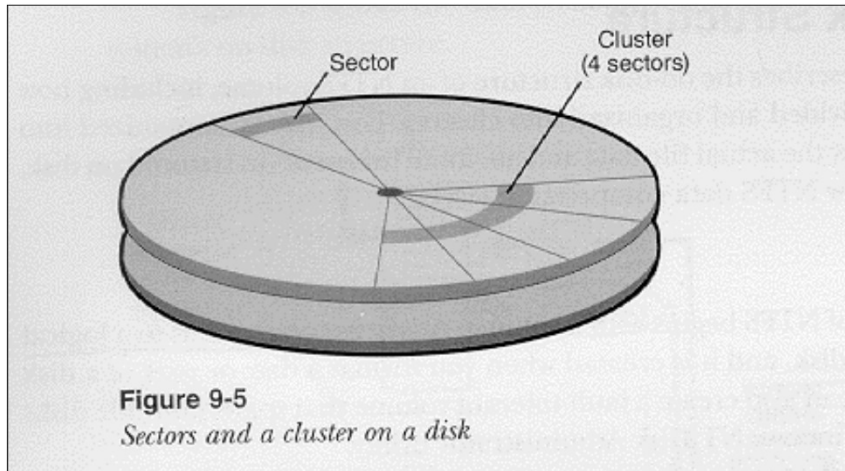
Exemple : Avec des clusters d'une taille 32Ko, la taille maximale d'une partition FAT est donc de 2Go.

MBR	FAT	FAT MIROIR	Fichiers
------------	------------	-------------------	-----------------



5.1 – Mémoires Disques : SGF - FAT 32

- Ils utilisent une **table d'allocation de fichiers**



FAT32 est un système 32 bits
 2^{28} , soit 268 435 455 blocs

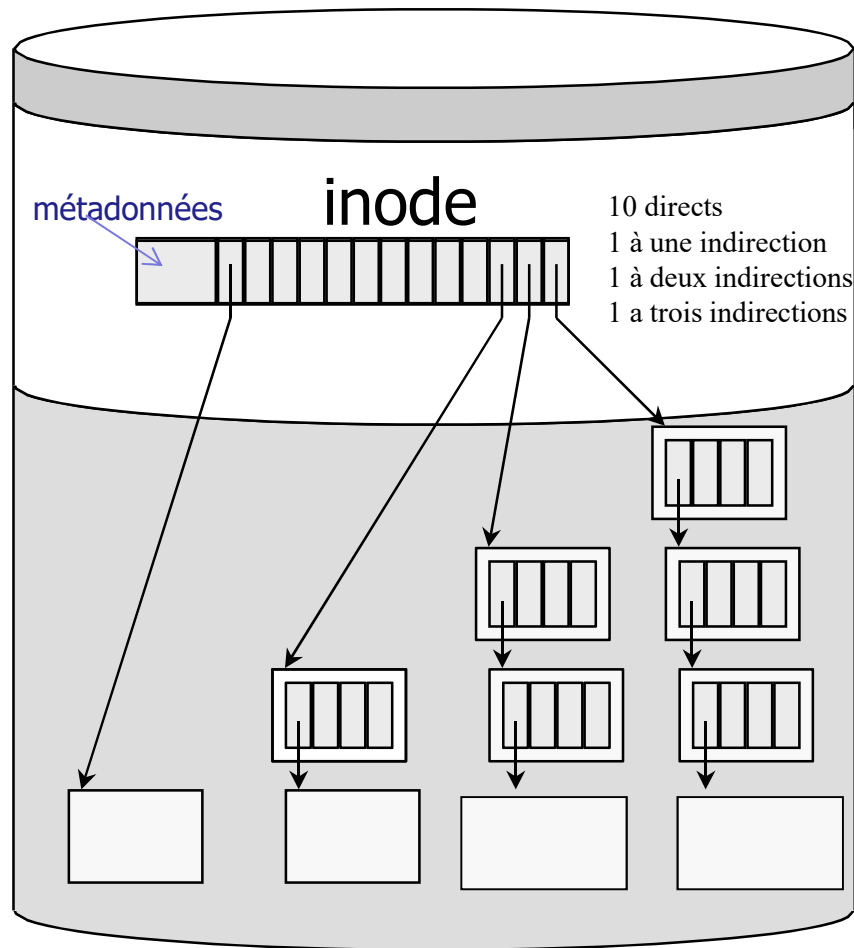
*la taille théorique maximum d'une partition FAT32 est de 8 To, toutefois
Microsoft la limite volontairement à 32 Go*

MBR	FAT	FAT MIROIR	Fichiers
-----	-----	------------	----------



5.2 – Mémoires Disques : SGF : System V d'Unix

Superbloc	Table Allocation des Blocs	Table d'allocation des inodes	Fichiers
-----------	----------------------------	-------------------------------	----------



Les 10 premiers champs (sur les 13) contiennent les adresses des 10 premiers blocs de données du fichier

La simple indirection, utilisée par le champ 11
La double indirection, utilisée par le champ 12
La triple indirection, utilisée par le champ 13

Le champ 11 pointe vers 256^1 blocs de données
Le champ 12 pointe vers 256^2 blocs de données
Le champ 13 pointe vers 256^3 blocs de données

En supposant que les blocs ont comme taille 1024 octets (1 Ko, et que chaque adresse (dans le cas d'un bloc d'adresses) est stockée sur 32 bits (4 octets), chaque bloc d'adresses en contiendra 256.

Avec ces informations en main, calculer la taille maximale d'un fichier.



5.3 – Mémoires Disques : NTFS (*New Technology File System*)

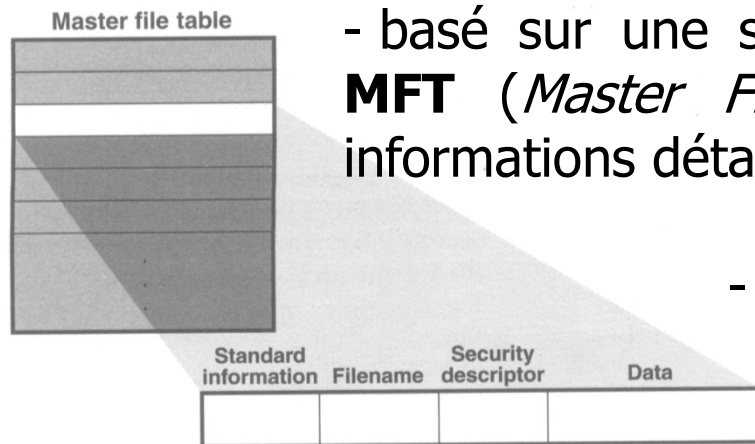


Figure 9-8
MFT record for a small file

- basé sur une structure appelée Table de Fichiers Maîtres **MFT** (*Master File Table*), permettant de contenir des informations détaillées sur les fichiers (sensible à la casse).

- Système d'adressage 64 bits

- La limite théorique de la taille d'une partition est de :
17 Exaoctets soit 17 milliards de To

*Le système de fichiers **NTFS s'inspire beaucoup des systèmes à base d'i-nodes**, mais en tentant de résoudre quelques uns de leur inconvénients. Notamment la surcharge induite par les fichiers de petites tailles.*

Organisation d'un disque NTFS

